# Streaming LLM Weights from CXL Memory: A Simulation Study of Regimes, Placement and Pipeline-Aware Pinning

Quang-Vinh Dang[a,*], Dat Le[b], Minh Ngoc Dinh[c]

[a]*British University Vietnam, Hung Yen, Vietnam*
[b]*Department of Science, Technology, and International Projects, Ho Chi Minh City University of Economics and Finance, Ho Chi Minh City 700000, Vietnam*
[c]*Millennia Education, Ho Chi Minh City, Vietnam*

**Abstract**

Models larger than GPU memory can be served by streaming weights from CXL memory. The GPU cannot address a CXL expander, so every transfer is a host-initiated DMA limited by the slower of the device and the PCIe link (10–20 GB/s). This is a simulation study: the link is modelled from published measurements, and only the simulator's overlap logic is checked on a real GPU with pinned host memory as the pool; no CXL device was measured. First, streaming is nearly free only above about 16k tokens per iteration: at 18 GB/s a 70B model is 1.10× slower than resident weights at 16k tokens and 91.7× slower at one token. Second, where resident layers sit matters more than which ones: contiguous resident sets idle the link when compute and transfer are comparable, while interleaved placements, such as a FlexGen-style per-layer split, stay within about 7% of a lower bound. Third, CAMP combines a work-conserving prefetcher with stall-cost-aware pinning (SCP), a plan-and-verify choice of resident layers. On Llama-2/3 and Gemma-2 CAMP is 1.05–2.82× faster than demand paging and within 1.014–1.069× of the lower bound. Against a tuned per-layer split it is ahead by up to 7%, behind by at most 3.2%, ties on uniform dense models and wins where reuse or unit sizes are non-uniform (up to 1.97× on encoder–decoder inference). Fourth, sharing one expander among $G$ GPUs raises the threshold

*Corresponding author. ORCID: 0000-0002-3877-8024
*Email addresses:* vinh.dq4@buv.edu.vn (Quang-Vinh Dang), datla@uef.edu.vn (Dat Le), minh.dinh@maeducation.com (Minh Ngoc Dinh)

$G$-fold, giving a simple sizing rule. A calibrated compute-time estimate changes latency by under 1%.

*Keywords:* Compute Express Link, memory disaggregation, LLM inference, prefetching, weight offloading, discrete-event simulation

---

## 1. Introduction

The weights of current large language models no longer fit in the memory of a single accelerator: Llama-3-70B needs 141 GB in FP16 and Llama-3.1-405B needs 812 GB, against 80 GB of HBM on a high-end GPU. The usual remedies are to shard the model over several GPUs, which multiplies the hardware bill, to quantise it, which changes the model, or to offload part of it to host DRAM, SSD or a remote tier and stream it back on demand [1–3]. Compute Express Link (CXL) memory expanders add a new option: byte-addressable DDR capacity attached to the host over PCIe-class links, at lower cost per gigabyte than HBM and without the capacity ceiling of the DIMM slots [4, 5]. The catch is bandwidth. Prototype Type-3 devices sustain on the order of 10–20 GB/s towards a GPU [6, 7], two orders of magnitude below HBM.

Two facts shape the problem and are easy to overlook when CXL memory is treated as a directly attached GPU memory tier. First, a CXL Type-3 expander is a CXL.mem endpoint behind the host root complex; a GPU cannot address it directly. Weights reach HBM through a host-initiated `cudaMemcpyAsync` from page-locked, CXL-backed host memory, so the achievable rate is set by the slower of the CXL device and the PCIe link, and every copy carries software and synchronisation costs [6]. Second, the access order of transformer inference is static, but the compute available to hide a transfer is not: it scales with the number of tokens processed per iteration. Streaming a layer of $p$ parameters costs $2p$ bytes of link traffic and enables $2pT$ FLOPs for $T$ tokens, so a link of 18 GB/s hides the transfer only when $T$ is in the tens of thousands (Section 3). Prefill of long prompts and very large batches are in that regime; interactive decode is not.

Known-graph tensor swapping is an established idea in training [8–12], and layer-wise weight streaming with overlapped I/O is established in inference [1, 7]. We do not claim these ideas as new. This paper asks what is left to decide, and how much it is worth, when LLM weights are streamed from

CXL through the host path. We make three kinds of contributions, and are explicit about which are positive and which are negative results.

*Contributions.*

1. **A path model, a threshold and a lower bound (Sections 3, 6.1).** A host-mediated transfer model with explicit initiator, bandwidth bound, per-copy and synchronisation costs and a staged (bounce-buffer) variant; a closed-form arithmetic-intensity threshold; an implementation-independent lower bound for whole-unit streaming; and a regime map showing that managed streaming comes within 1.10× of resident weights only above about 16k tokens per iteration (18 GB/s), while one-token decode is 91.7× slower. This part needs no policy and is the part most likely to transfer to real hardware.
2. **Where the policy gains come from (Sections 6.4, 6.3).** Against a conventional hot/cold baseline, nearly all of the benefit is explained by *placement*: contiguous resident sets idle the link when compute and transfer are comparable, and interleaved sets do not. A per-layer fractional split in the style of FlexGen is already interleaved and, in our simulator, stays within 3.2% of CAMP on uniform dense models once its buffer count is tuned. A baseline that adds one ingredient at a time (Section 6.3) shows which ingredients matter.
3. **The CAMP policy and the cases where it differs (Sections 4, 6.2, 6.5).** CAMP combines a work-conserving prefetcher with next-use admission, a pipeline planner and stall-cost-aware pinning (SCP), which scores candidate resident sets with the planner and verifies them in a dry run. SCP never lost more than 2.4% to the best of six simpler pin rules in 14 configurations and gained up to 22.3% over it, without being told which rule suits the workload. Against a FlexGen-style split whose buffer count and lookahead are tuned per configuration by simulation, CAMP is within ±3.2% on 16 of 18 rows and 7% faster on Gemma-2 (tied embedding); where per-layer reuse differs strongly (encoder–decoder) the split is 1.97× slower. Against the conventional hot/cold baseline the gain reaches 1.62×.
4. **A pool-sizing rule (Section 6.8).** When $G$ GPUs share one expander the per-GPU bandwidth falls by $G$ and so the threshold rises by $G$; streaming stays nearly free while $G \leq T/T^{\star}$. In simulation the rule is conservative by 0.85–1.00× in the transfer-bound cases, and CAMP

stays within 1.067× of the lower bound at every sharing level, so the rule tells an operator how many GPUs one expander can feed for a given iteration size.

5. **Methodology and negative results (Sections 5, 5.2, 6.6).** An event-driven simulator with common random numbers, ten seeds, a hardware check of its link and overlap logic on a real GPU, and a cost-model study. The compute estimate, a prominent component in related proposals, affects latency by at most 0.4% once calibrated by one probe iteration, and an LFU eviction policy is indistinguishable from LRU on cyclic scans.

*Nature of the evidence.* This is a modelling and simulation study, not a measurement of CXL hardware. All quantitative claims about CXL-backed serving are conditional on the link model of Section 3, whose bandwidth and fixed costs come from published prototype measurements and are swept from 8 to 64 GB/s (Section 6.7). The only measurements we made are on a PCIe-attached GPU reading page-locked host DRAM, and they test the simulator's copy and overlap logic, not CXL. Readers who need absolute numbers for a given expander should substitute their own measured bandwidth and per-copy cost into the closed-form threshold and lower bound, which is what Section 6.1 is designed for.

*Scope.* We target dense transformer models whose layer order is static. We do not evaluate Mixture-of-Experts routing, multi-tenant arbitration inside a CXL pool (Section 6.8 treats only equal sharing), KV-cache placement in CXL, or CXL switches. No CXL hardware was available: the CXL link is simulated with parameters anchored in published measurements, and the hardware validation (Section 5.2) uses pinned host memory on a PCIe-attached GPU as a stand-in for the pool. All GPU compute times other than the T4 micro-benchmark come from a roofline model, not from measurement.

## 2. Background and related work

*Reaching CXL memory from a GPU.* CXL expanders have been studied mainly as a CPU-side capacity tier: Pond [4] and TPP [5] place or migrate pages between local DRAM and CXL memory, and DirectCXL [13] and the characterisations of Melody [14] and Sun et al. [15] report latencies of roughly 140–410 ns and device-dependent bandwidth. Work on GPUs is more recent

and confirms the host-mediated nature of the path. TraCT [6] reports that a plain `cudaMemcpy` from a CXL-mapped region goes through a bounce buffer in host DRAM and must be avoided by page-locking the region so that the DMA engine reads the device directly; it measures 10.1 GB/s and 640 ns for a second-generation expander. ITME [7] prefetches model weights layer by layer from a CXL-hybrid memory into GPU memory with a DMA pipeline and reports about 18 GB/s for the prototype. LIA [16] offloads parameters to CXL memory in a single-GPU system and lets AMX-equipped host cores compute the layers that are cheaper to run where the data is, which removes weight traffic from the link for those layers; we study the complementary case in which all computation stays on the GPU and the question is how to schedule the transfers. DAK [17] instead lets the GPU read remote memory directly through the Tensor Memory Accelerator on NVLink-C2C and PCIe systems, and cache-coherent GPU–CPU links such as GH200 expose coherent memory to the GPU [18]; these are different access paths from the one we model. Compression raises the effective bandwidth of a CXL link [19] and is orthogonal to the policy studied here. Full-system CXL simulators with silicon validation, such as CXL-DMSim [20], model the device and host side in far more detail than we do but, to our knowledge, do not model GPU kernels.

*Weight offloading for LLM inference.* FlexGen [1] formulates the placement of weights, activations and KV cache over GPU, CPU and disk as a linear program with per-tensor percentages and overlaps I/O with compute for throughput-oriented generation; ZeRO-Inference/Infinity [2] stream partitioned weights from CPU and NVMe; llama.cpp [21] and vLLM [22] expose CPU offload of a fraction of the layers. LLM in a flash [3] streams weights from flash with windowing and row–column bundling to reduce bytes moved. PowerInfer [23] keeps frequently activated ("hot") neurons on the GPU and computes the rest on the CPU, exploiting activation sparsity rather than streaming weights; Fiddler [24] and MoE-Infinity [25] orchestrate CPU and GPU or cache experts by activation patterns for Mixture-of-Experts models. Serving systems batch iteration by iteration [22, 26], and chunked prefill [27] mixes prompt chunks with decode tokens, which is what makes the arithmetic intensity of an iteration a tunable quantity. KV-cache offloading [28, 29] and prefill/decode disaggregation [30] address different data and are complementary.

*Known-graph memory management.* When the future access order is known, Belady's optimal replacement [31] gives the offline reference. Training systems exploit the dataflow graph to swap activations and tensors between GPU and host or persistent memory: vDNN [8], SwapAdvisor [9] (joint scheduling, allocation and swap decisions by genetic search), AutoTM [10] (integer linear programming for placement and movement), Capuchin [11] (runtime tensor-access profiling) and Sentinel [12] (migration on heterogeneous memory); MAGIS and MEMO [32, 33] plan activation memory. Our setting differs in what is moved (read-only weights, no write-back), in the bandwidth regime (an order of magnitude below PCIe-attached DRAM for the same GPU) and in the access pattern (a cyclic scan with a regime set by the batch size), but the planning idea is the same, and we do not present known-graph prefetching itself as new. General tiering controllers infer behaviour from pressure or access samples [34, 35]. Request-level LLM simulators such as Vidur [36] model scheduling rather than the memory path. Performance models of LLM inference have also been used to optimise resource allocation: Sun et al. [37] validate models that predict inference time under a given placement of model blocks over geographically distributed servers, formulate block placement and request routing as an optimisation problem, and provide a CPU-only simulator of distributed LLM inference. Their resources are servers connected over the Internet and the decision is where blocks live; ours is one GPU behind a host-mediated memory tier and the decisions are which weights stay resident and when the others are transferred, so the two studies share the method (validated analytic models plus a simulator) rather than the system.

*Positioning.* Table 1 summarises the closest systems. CAMP differs from FlexGen in operating on whole units with an explicit transfer model and in choosing the resident set by a pipeline-aware objective and a verification step; our experiments show that the two are on par for uniform dense models and differ for non-uniform ones (Sections 6.2 and 6.5). We do not claim that frequency-based hot/cold partitioning is inferior in general.

## 3. System model

### *3.1. Host-mediated CXL-to-GPU path*

Figure 1 shows the path that the simulator models. The CXL Type-3 expander is exposed to the operating system as a CPU-less NUMA node. The CAMP runtime, a host thread, page-locks the CXL-backed region holding

Table 1: Closest prior systems and CAMP along the dimensions relevant to this paper (as described by their authors).

| System | Tier / path | Resident-set rule | Transfer scheduling | Evaluation |
|---|---|---|---|---|
| FlexGen [1] | GPU/CPU/disk, explicit copies | LP over weight/KV/activation fractions (per-tensor split) | overlapped I/O pipeline | real GPUs |
| PowerInfer [23] | GPU + CPU compute | hot neurons by profiled activation frequency | none (cold neurons run on CPU) | real consumer GPU |
| ITME [7] | CXL-hybrid memory → host → GPU | none described (layer-wise prefetch) | layer-wise DMA prefetch | hardware prototype |
| DAK [17] | GPU direct access (TMA) to remote memory | — | bypasses host prefetch | real NVLink-C2C / PCIe systems |
| vDNN, SwapAdvisor, AutoTM, Capuchin [8–11] | GPU ↔ host or NVM (training) | layer-wise, genetic search, ILP, or runtime profiling | prefetch or swap along the dataflow graph | real GPUs / servers |
| **CAMP** (this work) | host-mediated CXL → GPU, explicit path model | SCP: planner + dry-run verification | work-conserving, next-use admission, online cost model | event-driven simulation; link and overlap logic checked on a real GPU |

the weights and issues one `cudaMemcpyAsync` per layer unit on a dedicated copy stream; the GPU copy engine then reads the data through the root complex, which forwards the reads to the expander using CXL.mem, and writes it into the HBM slot of the target layer. Kernels of the consuming layer wait on a CUDA event recorded after the copy. The initiator is therefore the host, the data crosses two links in series (CXL, then PCIe), and the sustained rate is bounded by the slower of the two:

$$BW_{\text{eff}} = \eta \, \min(BW_{\text{CXL}}, \, BW_{\text{PCIe}}), \qquad t_{\text{copy}}(s) = t_0 + \left\lceil \frac{s}{c} \right\rceil t_c + t_{\text{sync}} + \frac{s}{BW_{\text{eff}}}, \tag{1}$$

where $s$ is the transfer size, $c$ the DMA chunk size, $t_0$ the per-copy API, driver and first-byte cost, $t_c$ the per-chunk descriptor cost and $t_{\text{sync}}$ the event synchronisation cost. If the region is *not* page-locked, the driver stages data through a bounce buffer in host DRAM; the CPU copy and the DMA are then serial and $BW_{\text{eff}} = (1/BW + 1/BW_{\text{stage}})^{-1}$ ("staged" mode). On a Tesla

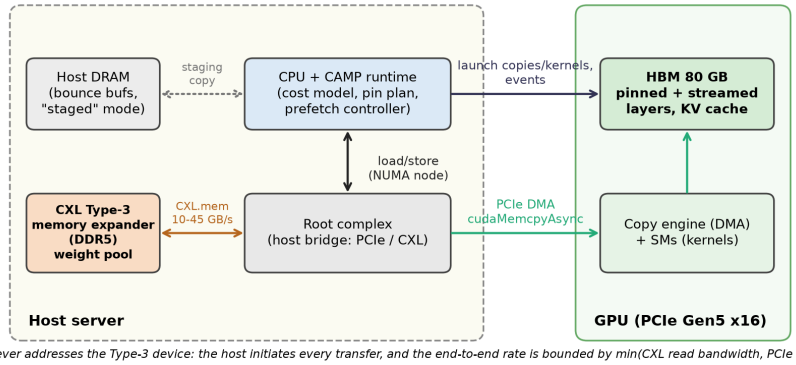



Figure 1: Host-mediated path modelled in this paper. The GPU does not address the CXL Type-3 expander; the CAMP runtime on the host issues DMA copies from page-locked CXL-backed memory into HBM, and synchronises kernels with events. "Staged" mode adds a bounce-buffer copy through host DRAM.

T4 we measured 12.4 GB/s for pinned and 4.6 GB/s for pageable copies, a ratio of 0.37 (Section 5.2), consistent with the bounce-buffer penalty reported for CXL regions [6].

The simulator additionally includes (i) a single non-preemptive copy engine that serves demand fetches before queued prefetches, (ii) log-normal noise on every copy and a slow autoregressive drift of the link bandwidth that models interference from other traffic, and (iii) a fixed host sampling gap between decode iterations during which the link keeps working. It does *not* model CXL switches or pooling across hosts, interleaving of several expanders (this is folded into $BW_{\text{eff}}$, which we sweep from 8 to 64 GB/s), GPU-initiated direct access to remote memory, cache-coherence traffic, or contention with host CPU memory traffic. Table 2 lists the parameters and their basis.

### *3.2. Layer units and iteration traces*

A model is decomposed into *units*, the granularity at which weights are fetched and pinned: one attention block (QKV and output projections and norms), one feed-forward block, the embedding table and the output head (tied models share one unit). Table 3 lists the configurations. An *iteration* is one pass over the units: a prefill pass over $B$ prompts of $S$ tokens, a decode step for $B$ sequences with context length $C$, or a mixed iteration in which a prefill chunk piggybacks on decode tokens. The embedding access is a gather of $T$ rows rather than a full-table fetch. The KV cache and workspace stay in

Table 2: Simulated platform. "Measured" entries are anchored on published device measurements or on our own T4 micro-benchmarks (Section 5.2); all values are exposed as parameters of the simulator.

| **Component** | **Value** | **Basis** |
|---|---|---|
| GPU | H100-class: 989 TFLOP/s FP16 dense, 3.35 TB/s HBM, 80 GB | vendor data sheet; true efficiencies (GEMM 0.62, memory 0.86, attention 0.42) are hidden from CAMP |
| Host-to-GPU path | pinned DMA via root complex ("direct"); optional bounce-buffer path ("staged") | [6]; staged/pinned ratio 0.37 measured on T4 |
| Effective CXL→GPU bandwidth | 18 GB/s (default, "measured"); 45 GB/s ("expected", x16 Gen5); sweep 8–64 GB/s | ≈10 GB/s device-level [6], ≈18 GB/s DMA to GPU [7]; PCIe Gen5 pinned copies are the upper bound |
| Per-copy fixed cost | 12 $\mu$s API/driver/first-byte + 6 $\mu$s event sync + 3 $\mu$s per 32 MB chunk (≈21 $\mu$s for a 64 MiB copy) | T4: fitted intercept of copy time vs. size is 10.6 $\mu$s (the extra simulator terms are $< 0.1\%$ of a 64 MiB copy); CXL.mem load latency ≈0.3–0.6 $\mu$s [6, 14] |
| Staging hop (staged mode) | 8 GB/s CPU copy, serial with DMA | T4 pageable-copy measurement (4.6 vs. 12.4 GB/s) |
| Noise | per-copy log-normal $\sigma = 0.06$; slow AR(1) bandwidth drift $\sigma = 0.05$ ($\tau \approx 2.5$ s); per-layer compute $\sigma = 0.03$ | varied up to 8× in Section 6.10 |
| Host sampling gap | 150 $\mu$s between decode iterations (link keeps working) | typical scheduler/sampler cost |

HBM and reduce the capacity $C_w$ available for weights: for Llama-3-70B with $B$=16 sequences of up to 4k tokens, the KV cache takes 21.5 GB, leaving $C_w = 54.5$ GB (39% of the weights).

### *3.3. Compute model and regimes*

The execution time of a unit on the simulated GPU follows a roofline model [38] per sub-operation:

$$t = \max\Big(\frac{F_g}{P\,\eta_g},\ \frac{B_g}{W\,\eta_m}\Big) + \max\Big(\frac{F_a}{P\,\eta_a},\ \frac{B_a}{W\,\eta_m}\Big) + n_k\,t_{\text{launch}}, \tag{2}$$

where $F_g, B_g$ are the FLOPs and HBM bytes of the projections, $F_a, B_a$ those of the attention core (including the KV cache read), $P$ the peak FLOP rate, $W$ the HBM bandwidth, $\eta$ the efficiencies and $n_k$ the number of kernels. This

Table 3: Model configurations used in the evaluation (public model-card hyper-parameters; FP16 weights). Layer units are one attention block, one feed-forward block, the embedding table and the output head.

| Model | Params (B) | Layers | $d$ | Heads/KV | $d_{ff}$ | Vocab | Embed. | Weights (GB) |
|---|---|---|---|---|---|---|---|---|
| llama2-7b | 6.74 | 32 | 4096 | 32/32 | 11008 | 32k | untied | 13.5 |
| llama2-13b | 13.02 | 40 | 5120 | 40/40 | 13824 | 32k | untied | 26.0 |
| llama3-8b | 8.03 | 32 | 4096 | 32/8 | 14336 | 128k | untied | 16.1 |
| llama3-70b | 70.55 | 80 | 8192 | 64/8 | 28672 | 128k | untied | 141.1 |
| llama3-405b | 405.85 | 126 | 16384 | 128/8 | 53248 | 128k | untied | 811.7 |
| gemma2-9b | 9.24 | 42 | 3584 | 16/8 | 14336 | 256k | tied | 18.5 |

is the *ground truth* of the simulated device; CAMP sees only the features $(F_g, B_g, F_a, B_a, n_k)$ and must estimate $t$ (Section 4.1).

Streaming a unit of $p$ parameters moves $b\,p$ bytes ($b = 2$ for FP16) and enables $2pT$ FLOPs, so compute hides the copy when

$$T \;\geq\; T^{\star} = (1-\varphi)\,\frac{P\,\eta_g}{BW_{\text{eff}}} \cdot \frac{b}{2}, \tag{3}$$

with $\varphi$ the fraction of weight bytes kept resident. For $P\eta_g = 613$ TFLOP/s, $\varphi = 0.4$ and $BW_{\text{eff}} = 18$ or 45 GB/s, $T^{\star} \approx 20$k or 8k tokens per iteration. Decode with $B \leq 512$ is therefore always link-bound, and the only lever is the number of bytes that cross the link per iteration. Section 6.1 confirms this threshold in the simulator.

### *3.4. A lower bound*

An iteration cannot finish before its compute-only time $\sum_u t_u$, nor before the minimum number of streamed bytes has crossed the link. If any unit is streamed, one slot as large as the largest streamed unit must stay free, so with $W_{\text{tot}}$ total weight bytes and cache capacity $C_w$ at least $W_{\text{tot}} - (C_w - s)$ bytes are streamed, where $s$ is the size of the largest streamed unit (all larger units must then be pinned and fit in $C_w - s$). We take the smallest feasible $s$:

$$T_{\text{LB}} = \max\Big(\sum_u t_u,\;\; \frac{\max\{0,\, W_{\text{tot}} - C_w + s_{\min}\}}{BW_{\text{eff}}}\Big). \tag{4}$$

The bound ignores per-copy overheads and pipeline fill, and is exact in the link-bound regime for uniform units. It is implementation-independent for policies that move and pin *whole* units: none can beat it, and it gives each result an absolute reference. A policy that splits every layer fractionally, as FlexGen's per-layer percentages do, can undercut it slightly (by up to 3.1% in our experiments), because the streaming slot then holds a fraction of a unit.

## 4. The CAMP policy

CAMP has three parts. All rely on the static unit order of the model (the "graph"), none on offline profiling of the model or workload.

### *4.1. Runtime cost model*

The prefetcher and the pinning stage need an estimate $\hat{t}_u$ of each unit's compute time for the *current* batch descriptor $(B, S, C)$. CAMP computes it from tensor shapes with the roofline of Eq. (2), using generic constants for the device class (efficiencies $\hat{\eta}_g = 0.50$, $\hat{\eta}_m = 0.80$, $\hat{\eta}_a = 0.35$) and the peak rates that the driver reports. These constants differ from the device's true efficiencies, so the estimate is calibrated online: for each unit kind $\kappa$ (attention, feed-forward, embedding, head) the runtime keeps a multiplicative correction updated by an exponential moving average of the ratio between observed and nominal duration,

$$\hat{c}_\kappa \leftarrow (1-\alpha)\,\hat{c}_\kappa + \alpha\,\frac{t_u^{\mathrm{obs}}}{t_u^{\mathrm{nom}}}, \qquad \hat{t}_u = \hat{c}_\kappa\, t_u^{\mathrm{nom}}, \quad \alpha = 0.3. \tag{5}$$

The first iteration therefore runs with the generic estimate, and every later iteration, or any re-planning after the first probe iteration, uses the calibrated one. The current layer's own duration is *not* needed before it completes: the estimate of unit $u$ is available from its features before it runs, and the observation after it runs only refines future estimates. We quantify the effect of estimator error in Section 6.6.

### *4.2. Work-conserving prefetching with next-use admission*

Because the unit order is static, the next use of every unit is known. Algorithm 1 issues prefetches whenever the link or a layer completes. A fetch of the unit needed at position $j$ is admitted if free capacity exists or if some resident, unpinned unit has its next use *after* $j$ (Belady-style admission [31]); otherwise the scan stops. The demand fetch of the layer about to run is always issued first, and prefetches cannot take capacity while a demand fetch is waiting for room. This controller is work-conserving: it never leaves the link idle if capacity allows another useful fetch. An earlier formulation limited the lookahead to the units whose cumulative transfer time fits in the estimated remaining compute of the current layer; in all our workloads that gate never binds, because completions re-trigger the controller, and results are identical with and without it (Section 6.6). The estimate $\hat{t}_u$ is thus not on the critical path of prefetching; it matters for pinning.

**Algorithm 1** Work-conserving prefetch controller (invoked at layer start, layer begin and every copy completion)

**Input:** access order $Q$, current position $i$, cache with pinned set $\mathcal{P}$
$j \leftarrow i + 1$
**while** $j < |Q|$ **and** no demand fetch is blocked **do**
    $u \leftarrow Q[j]$
    **if** $u$ is neither resident nor in flight **then**
        **if** free capacity $\geq$ size($u$) **then**
            issue prefetch($u$)
        **else**
            $v \leftarrow \arg\max_{w \text{ resident}, w \notin \mathcal{P}, w \neq Q[i]}$ nextuse($w$)
            **if** nextuse($v$) $> j$ **then** evict $v$; issue prefetch($u$) **else break**
            **end if**
        **end if**
    **end if**
    $j \leftarrow j + 1$
**end while**

*4.3. Stall-cost-aware pinning*

*Problem statement.* Let $U$ be the set of units with sizes $s_u$, $C_w$ the weight-cache capacity, $\mathcal{P} \subseteq U$ the pinned set and $T_{\text{plan}}(\mathcal{P})$ the predicted steady-state iteration time of the pipeline planner defined below. SCP solves

$$\min_{\mathcal{P} \subseteq U} \; T_{\text{plan}}(\mathcal{P}) \quad \text{s.t.} \quad \sum_{u \in \mathcal{P}} s_u + \max_{v \in U \setminus \mathcal{P}} s_v \;\leq\; C_w, \tag{6}$$

that is, the pinned units plus one streaming slot large enough for the largest streamed unit must fit. This is *not* a 0–1 knapsack: $T_{\text{plan}}$ is not additive over units and depends on where the pinned units sit in the sequence. The additive proxy

$$\max_{\mathcal{P}} \; \sum_{u \in \mathcal{P}} F(u)\, t_{\text{copy}}(s_u) \quad \text{s.t.} \quad \sum_{u \in \mathcal{P}} s_u \leq C_w - s^*, \tag{7}$$

with $F(u)$ the number of accesses of $u$ per iteration, is a 0–1 knapsack whose benefit-per-byte is $F(u)\, t_{\text{copy}}(s_u)/s_u \approx F(u)/BW_{\text{eff}}$. For uniform sizes it ranks units by $F(u)$ alone, which is the frequency rule of CAMP-v1 and of hot/cold partitioning. We solve Eq. (7) exactly by dynamic programming and use it as a baseline and as one SCP candidate; it is a proxy, and Section 6.4 shows when it is a poor one.

*Pipeline planner.* $T_{\text{plan}}(\mathcal{P})$ is evaluated with a deterministic recurrence over one iteration repeated three times. Let $\hat{c}_j$ be the estimated compute time of access $j$, $L$ the time at which the link becomes free and $R = C_w - \sum_{\mathcal{P}} s_u$ the streaming buffer. A streamed access $j$ is fetched as early as the link is free and the buffer has room, i.e. once earlier streamed units have finished computing and released enough bytes; it then computes when both the previous layer and its own fetch have finished:

$$f_j = \max(L,\ \rho_j) + t_{\text{copy}}(s_j), \qquad L \leftarrow f_j, \tag{8}$$

$$b_j = \max(e_{j-1},\ f_j), \qquad e_j = b_j + \hat{c}_j, \tag{9}$$

where $\rho_j$ is the earliest time at which the buffer holds at most $R - s_j$ bytes of streamed units, and pinned accesses skip the fetch. The planner costs $O(|Q|)$ per evaluation.

*Search and verification.* Algorithm 2 shortlists structured candidates (no pinning, prefix, frequency-ranked, knapsack-DP, evenly interleaved "stride" sets at five pinned fractions, and the lazy marginal-gain greedy set), scores them with $T_{\text{plan}}$, refines the best by add/remove/local-swap search and a short simulated-annealing polish (about 1 200 planner evaluations), and finally replays the shortlisted sets in the event engine for three iterations *using the cost model's estimated layer times*, never the device's true times, and keeps the fastest replay. The replay removes planner approximations, such as ignoring buffer reuse across iterations, from the final decision. Planning takes 0.2–0.7 s for 160-unit models on one core and is performed once per operating point (or after calibration), not per token.

*Why placement matters.* Consider $N$ equal units with compute $c$ and fetch time $f > c$, a streaming buffer holding $m \geq 2$ units, and $P$ pinned units. If the pinned units form a contiguous prefix, the link can only run $m - 1$ units ahead while they compute, and idles for the rest:

$$\begin{aligned} T_{\text{prefix}} &\approx (N - P)\, f + \max\{0,\ P\, c - (m-1) f\}, \\ T_{\text{interleaved}} &\approx \max\{(N - P)\, f,\ N\, c\}. \end{aligned} \tag{10}$$

The penalty vanishes when $c \ll f$ (decode), and is largest when compute and transfer are comparable. Frequency-ranked and prefix pinning, which break ties by sequence position, generate exactly the contiguous case; Section 6.4 measures the link utilisation they lose.

**Algorithm 2** Stall-cost-aware pinning (SCP)

**Input:** unit sequence with sizes, cost model $\hat{t}$, link model $t_{\text{copy}}$, capacity $C_w$
$\mathcal{S} \leftarrow \{\emptyset\} \cup \{\textsc{Stride}(g) : g \in \{.3, .45, .6, .75, .9\}\} \cup \{\textsc{Prefix}, \textsc{Freq}, \textsc{KnapsackDP}, \textsc{Greedy}\}$ ▷ feasible sets only
$\mathcal{P}^* \leftarrow \arg\min_{\mathcal{P} \in \mathcal{S}} T_{\text{plan}}(\mathcal{P})$
$\mathcal{P}^* \leftarrow \textsc{LocalSearch}(\mathcal{P}^*)$ ▷ add / remove / swap within $\pm 3$ positions
$\mathcal{P}^* \leftarrow \textsc{Anneal}(\mathcal{P}^*,$ 1200 evaluations)
$\mathcal{S} \leftarrow \mathcal{S} \cup \{\mathcal{P}^*\}$
**for** $\mathcal{P} \in \mathcal{S}$ **do**
    $\tau(\mathcal{P}) \leftarrow$ replay 3 iterations in the event engine with estimated layer times
**end for**
**return** $\arg\min_{\mathcal{P} \in \mathcal{S}} \tau(\mathcal{P})$

*Transient layers and eviction.* Units that are not pinned are evicted by next-use order (furthest next use first), which is optimal for the known order [31]; the admission rule of Algorithm 1 prevents a prefetch from displacing a nearer use. On a cyclic scan with equal frequencies, LRU, FIFO and LFU all evict the layer that will be needed soonest and retain nothing (Section 6.5).

## 5. Evaluation methodology

### *5.1. Simulator*

*Why simulation.* A GPU cannot address a CXL expander, so a real experiment needs a server with a CXL Type-3 device, a PCIe-attached GPU on the same root complex, and a runtime that places weights in the CXL-backed memory. We had no such platform. A simulator lets us explore bandwidths from 8 to 64 GB/s, models up to 405B parameters and many policies under identical conditions, which no single testbed would allow, at the price that every result inherits the accuracy of the link and compute models.

The simulator (`camp_sim`, released with this paper) is an event-driven model of one GPU, one copy engine and an HBM weight cache. It contains no policy: prefetch, victim selection and pinning are plug-ins. The compute stream executes accesses strictly in order; each access first issues its demand fetch, then lets the policy issue lookahead, then waits for the weights. Copy

completions and layer completions are events, and policies are re-invoked at both. Noise is drawn from a generator keyed by (seed, event identity), and the bandwidth drift is indexed by simulated time, so different policies run with the same seed see the same environment for the same copy or layer (common random numbers); policy comparisons use paired seeds. All experiments use ten seeds unless stated; we report means and 95% confidence half-widths ($t$-distribution; at most 1.9% of the mean for CAMP-v2 in Table 5). Layer weights are never materialised: the simulator tracks bytes, not tensors.

Three things are checked independently of the experiments. (i) the test suite (`tests/test_engine.py`) compares the deterministic engine with closed-form pipeline times (demand paging $N(f + c)$; prefetch with a ring buffer $N \max(f, c)$; pinned layers $(N - P)f$; compute-bound $Nc$; non-uniform unit sizes; next-use retention on a cyclic scan; noise reproducibility), which agree to within 2%. (ii) The analytic planner agrees with the engine to a mean absolute error of 0.18% (maximum 3.3%) over 80 random model/workload/cache/link configurations (Section 6.10). (iii) The link and overlap logic is compared with a real GPU (Section 5.2).

*5.2. Hardware check of the link and overlap logic*

The simulator's novel parts are the copy path and the overlap of transfers with compute. We check them on a real GPU with CUDA streams and events (`validation/hw_validate.py`, run on a Tesla T4, PCIe Gen3 x16, CUDA 13). *This is a PCIe-attached GPU reading page-locked host DRAM: no CXL device was measured*, and the check says nothing about CXL device latency or bandwidth, which we take from the literature. Twenty-four weight tensors of 64 MiB live in page-locked host memory, standing in for the pool; layer $u$ computes a GEMM of $T$ tokens against its tensor. The loop supports a configurable prefetch depth (ring of device slots, events for slot reuse and copy completion) and a number of tensors pinned in HBM. We measure, *in the same loop*, (a) a compute-only iteration (all tensors resident) for each $T$, (b) a copy-only iteration, and (c) 30 composite configurations (five $T$ values $\times$ six combinations of depth and pinned count), three repetitions each. The simulator is then composed from the separately measured compute time per unit and the fitted copy cost (12.4 GB/s, 10.6 $\mu$s) and asked to predict the composites. No parameter is fitted to the composites.

Table 4 and Figure 2 give the result, and it is mixed. Across the 30 configurations the mean absolute error is 7.6% (median 5.6%, maximum 25%). In the link-bound regime (prefetch depth $\geq 1$, $T \leq 1024$, no pinned tensors;

Table 4: Simulator versus a real GPU (Tesla T4, PCIe Gen3 x16; 24 weight tensors of 64 MiB streamed from pinned host memory with a fixed-depth ring). The simulator is composed from separately measured compute-only and copy parameters and asked to predict the measured composite iteration time; no parameter is fitted to the composites. The last column gives the error of the closed-form expression $\max(n_s t_{copy}, N t_{comp})$ (demand paging: $n_s t_{copy} + N t_{comp}$), which uses the same measured inputs but no simulator.

| Regime | Configs | MAPE (%) | Signed (%) | Max \|err\| (%) | Closed-form MAPE (%) |
|---|---|---|---|---|---|
| overlapped, link-bound (compute $< 0.3\times$ copy) | 10 | 3.9 | +3.9 | 24.0 | 1.5 |
| overlapped, balanced (0.3–1.2×) | 10 | 8.8 | -2.8 | 15.5 | 7.8 |
| overlapped, compute-bound ($> 1.2\times$) | 5 | 5.6 | -5.6 | 6.3 | 5.6 |
| demand paging (serial copy, compute) | 5 | 14.6 | +14.3 | 25.5 | 14.2 |
| all configurations | 30 | 7.6 | +1.8 | 25.5 | 6.4 |
| with pinned units only | 10 | 9.7 | | | |

nine configurations) the error is 0.6% on average and at most 1.3%, which supports the copy path, queueing and overlap logic that every later result depends on. The error grows where compute time per unit is comparable to or larger than copy time, and its sign differs between regimes: under demand paging the simulator over-predicts by up to 25% (the measured serial loop is faster than the sum of the separately measured copy and compute times), and in overlapped, compute-bound configurations ($T \geq 2048$) it under-predicts by -9% on average (compute in the composite loop is slower than in the compute-only loop). We did not isolate the cause; candidate explanations such as GPU power or clock state differing between back-to-back and interleaved execution, or driver-level interaction between copies and kernels, are untested. Configurations with pinned tensors have a mean error of 9.7%. Two limits of this check should be stated plainly. First, a closed-form expression with the same measured inputs, $\max(n_s t_{\text{copy}}, N t_{\text{comp}})$ for overlapped runs and $n_s t_{\text{copy}} + N t_{\text{comp}}$ for demand paging, has a mean error of 6.4% over the same configurations, so on this simple loop the simulator is no more accurate than algebra; the check validates its inputs and overlap semantics, not its additional machinery (non-uniform units, queueing, noise), which is covered by the tests above. Second, the loop uses a fixed-depth ring, not CAMP's work-conserving policy, so the policy itself is not exercised on hardware. Pageable (staged) copies reached 0.37 of the pinned bandwidth (4.6 versus 12.4 GB/s), which implies a staging-hop rate of 7.3 GB/s in the serial two-hop model.

*Consistency with a published CXL weight-streaming result.* The only published end-to-end weight-streaming measurement from a CXL prototype that we

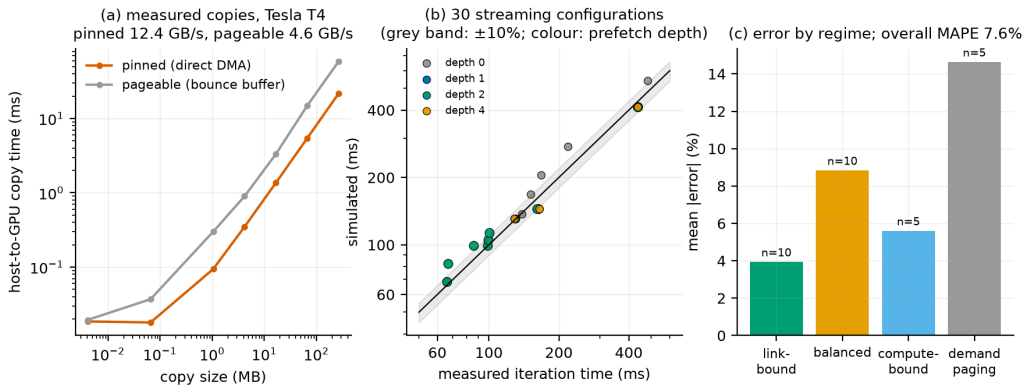



Figure 2: Check on a Tesla T4 (PCIe-attached GPU, pinned host memory as the pool). (a) Measured host-to-GPU copy time versus size for pinned and pageable memory. (b) Simulated versus measured iteration time for the 30 streaming configurations. (c) Mean absolute error by regime; the ratio of compute to copy time per unit defines the regime.

found is ITME [7]: an A100-80GB, Llama-3.1 models, a prototype delivering 18 GB/s, a workload of 16 prompts of 4k tokens, and a prefetch buffer of a few GB with a three-deep pipeline. The authors report that weight streaming stays within 1–5% of a host-memory baseline, and within 8% with a 4 GB buffer. We configured the simulator for the same GPU, link rate and workload (all weights streamed, one to three buffer slots, 18 and 22 GB/s) and obtained an overhead of 0.4–0.6% over compute-only execution for the 70B model and 0.7–2.0% for the 8B model. At 65k tokens per iteration the threshold of Eq. (3) for full streaming is $T^\star \approx 10.7$k tokens, so the model predicts a compute-bound iteration, as the reported 1–5% deltas indicate. We do not claim a quantitative match: the reported figures compare against a host-DRAM baseline rather than compute-only execution, we lack the prototype's exact pipeline, and the 8% reported for the smallest buffer is larger than any value our model produces. The check shows that the regime prediction is consistent with a measurement on a real CXL device, which our own hardware check cannot show.

### *5.3. Workloads, baselines and metrics*

Workloads use the models of Table 3. Large-model configurations ("main") are Llama-3-70B and Llama-3-405B with the HBM left after the KV cache of the workload as weight cache: decode with $B = 16$ sequences and 2k-token contexts (KV capacity reserved for 4k), mixed iterations with 32 decode

tokens plus a 512-token prefill chunk, and prefill of four 2k- or 4k-token prompts. "Constrained" configurations give a 13B or 9B model 40% of its weight bytes as cache, as on a GPU shared with other tenants. Non-uniform reuse is evaluated with Gemma-2-9B (tied embedding and head), speculative decoding (13B target, 1B draft, $k = 4$ draft steps per verification) and a 24+24-layer encoder–decoder processing 64 output tokens per request.

Baselines are given the same cache capacity and link, and the same tuning effort that CAMP does not need:

- **Demand+LRU**: no prefetch (the behaviour of an unmanaged mapping).
- **Static-$k$**: fixed lookahead $k \in \{1, 2, 4, 8\}$ chosen per configuration, LRU, no pinning. **Reactive** (lookahead grows on observed stalls, TMO-like [34]) and **Aggressive-10** (fixed deep lookahead, Limoncello-like [35]) behave as Static-$k$ without pins (within 8.2%) and are listed only in `results/e2_main.json`.
- **FlexGen-style**: a uniform fraction of *every* layer stays resident, the rest is streamed through $n$ buffers of the largest unit with lookahead $k$, FIFO eviction [1]; the fraction follows from $n$, and $(n, k)$ is chosen per configuration by simulation from ten candidates (a stand-in for FlexGen's policy search). The split is modelled at fractional granularity, finer than CAMP's whole units.
- **Hot/Cold**: resident set chosen by access frequency, ties broken at random, up to capacity minus one streaming slot ("equal budget"), streaming with tuned lookahead (PowerInfer-style partitioning [23] adapted to streaming).
- **Stride**: evenly interleaved resident set at equal budget with CAMP's prefetcher; this isolates the value of interleaving without the planner.
- **CAMP-v1**: the earlier formulation, with lookahead gated by the compute estimate and frequency pinning at the conventional two-slot reserve, and **CAMP-v2** (this paper), with the cost model calibrated after one probe iteration and SCP with dry-run verification; variants with the generic (uncalibrated) and the exact device model are reported where relevant.

These are reimplementations of the *mechanisms* in the common simulator, not ports of the original systems. Section 6.3 adds one ingredient at a time to a conventional hot/cold baseline to show which differences matter.

The metric is the steady-state iteration latency (the first iteration is a warm-up and excluded). *Speed-up vs. demand paging* is the demand-paging latency divided by CAMP-v2's; *speed-up vs. best external* is the best of Demand, Static-$k$, FlexGen-style, Hot/Cold and CAMP-v1 divided by CAMP-v2's (so values below one mean CAMP is slower); *det./LB* is the noise-free CAMP-v2 latency divided by $\max(T_{\mathrm{LB}}, \sum_u t_u)$ with the bound of Eq. (4).

## 6. Results

### *6.1. Regime map: when does streaming from CXL pay off?*

Figure 3 gives the latency of CAMP-managed streaming relative to a GPU with all weights resident, for Llama-3-70B with 40% of the weights cached, as a function of tokens per iteration and of $BW_{\mathrm{eff}}$. At 18 GB/s the slowdown is 91.7× for one token, 90.9× at 64 tokens, 77.2× at 256, 20.4× at 1 024, 5.0× at 4 096, 1.10× at 16 384 and 1.01× at 65 536. At 45 GB/s the same rows read 36.0×, 35.7×, 30.3×, 8.0×, 1.95×, 1.00× and 1.00×. The transition follows Eq. (3), $T^{\star} \approx 20$k tokens at 18 GB/s and 8 000 at 45 GB/s. Prefill of long prompts and very large batches can therefore run from CXL-resident weights at close to resident speed, so capacity can be traded for little latency. Interactive decode cannot: with at most 256 tokens per iteration the slowdown is 77.2–91.7× at 18 GB/s, and the only lever is to move fewer bytes. Neither prefetching nor cleverer replacement changes this; a larger cache, a faster link, quantisation (Section 6.9) or compression [19] do. Across the whole map CAMP is 0.995–1.064× the lower bound (median 1.010) and 1.12–2.49× faster than demand paging.

### *6.2. Main comparison*

Tables 5 and 6 and Figure 4 compare the policies in 18 rows (9 workloads at two link bandwidths). Four observations.

CAMP-*v2 is close to the lower bound everywhere.* Without noise its latency is 1.002–1.069× the bound of Eq. (4) (median 1.014), and with the default noise 0.996–1.068× (values just below one are noise around a deterministic bound). The largest gaps are in the balanced 4k-token prefill configurations at 18 GB/s.

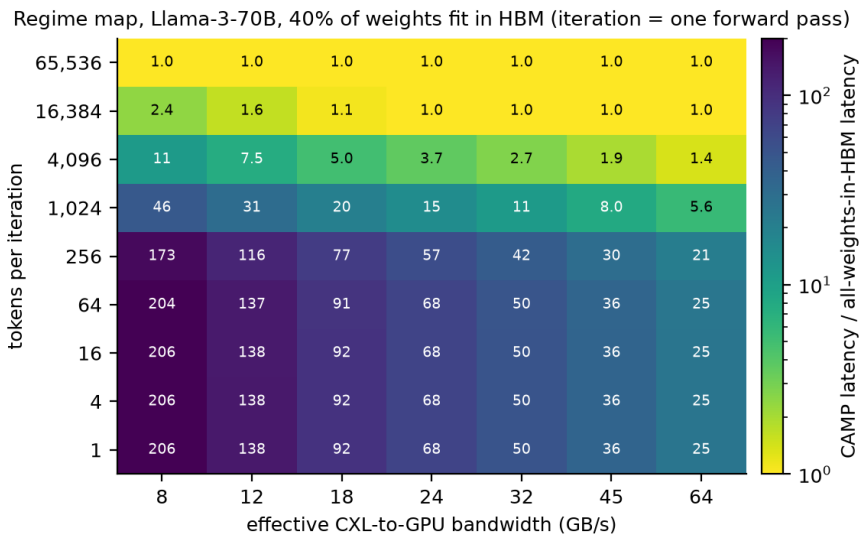



Figure 3: Regime map: CAMP iteration latency relative to all-weights-in-HBM, Llama-3-70B with 40% of the weights cached, as a function of tokens per iteration and effective CXL-to-GPU bandwidth (each cell: four seeds).

*Against demand paging the gain is large; against the strongest baseline it is not.* CAMP-v2 is 1.05–2.82× faster than demand paging (median 1.66; 1.63–2.37× in the constrained rows), 1.01–2.01× faster than the best fixed-lookahead prefetcher, up to 1.62× faster than the equal-budget Hot/Cold baseline and up to 1.38× faster than CAMP-v1. The best of the five external baselines, which on the large models is the tuned FlexGen-style split, is within 1% of CAMP-v2 in 14 of 18 rows; CAMP-v2 is faster by up to 7% in 2 rows (both Gemma-2) and slower by up to 3.2% in 2 rows (13B and 70B prefill at 18 GB/s). On the 70B and 405B configurations alone, CAMP-v2 is within 0.99–1.00× of the best external baseline, i.e. it ties it. A first version of the FlexGen-style baseline fixed two buffers and a lookahead of one or two layers and was markedly slower than CAMP in several rows; tuning its buffer count and lookahead removed most of the gap, and we report the tuned baseline.

*Interleaving, not the planner, explains most of the gain over a conventional baseline.* The stride baseline, which pins evenly spaced units and uses CAMP's prefetcher but has no planner, is slower than CAMP-v2 by more than 1% in 10 of 18 rows and by up to 22%. The remaining difference between stride pins and SCP is largest in balanced prefill at 18 GB/s (Section 6.3).

Table 5: Main comparison on the large-model configurations that do not fit one 80 GB GPU. Iteration latency in ms (mean of 10 seeds with common random numbers; best entry per row in bold). "Cache" is the share of weight bytes that fit in the HBM weight cache. FlexGen = per-layer fractional split with buffer count and lookahead tuned per row; Hot/Cold and Stride = equal-budget pinning with CAMP's or a tuned fixed lookahead (Section 5.3); LB = lower bound of Section 3.4; det./LB = noise-free CAMP-v2 latency divided by LB; "vs. best ext." = best of Demand/Static-$k$/FlexGen/Hot-Cold/CAMP-v1 divided by CAMP-v2.

| Model | Workload | Cache | GB/s | Demand | Static-$k$ | FlexGen | Hot/Cold | Stride | CAMP-v1 | CAMP-v2 | LB | det./LB | vs. Demand | vs. best ext. |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Llama-3-70B | dec. $B$=16,2k | 39% | 18 | 7 853 | 7 800 | 4 896 | 4 875 | 4 891 | 5 065 | **4 870** | 4 772 | 1.012 | 1.61× | 1.00× |
| | dec. $B$=16,2k | 39% | 45 | 3 182 | 3 127 | 1 924 | 1 920 | 1 931 | 1 993 | **1 916** | 1 909 | 1.016 | 1.66× | 1.00× |
| | mixed 32+512 | 23% | 18 | 7 931 | 7 800 | 6 133 | **6 095** | 6 151 | 6 193 | **6 095** | 5 965 | 1.013 | 1.30× | 1.00× |
| | mixed 32+512 | 23% | 45 | 3 260 | 3 127 | 2 438 | 2 436 | 2 445 | 2 463 | **2 426** | 2 386 | 1.019 | 1.34× | 1.00× |
| | pre. $B$=4 $S$=2k | 52% | 18 | 9 704 | 7 810 | **3 870** | 4 764 | 3 968 | 4 682 | 3 885 | 3 728 | 1.032 | 2.50× | 1.00× |
| | pre. $B$=4 $S$=2k | 52% | 45 | 5 025 | 3 119 | 1 908 | 2 660 | 2 192 | 2 408 | **1 907** | 1 887 | 1.006 | 2.64× | 1.00× |
| | pre. $B$=4 $S$=4k | 50% | 18 | 11 667 | 7 818 | **4 079** | 6 201 | 5 066 | 5 720 | 4 141 | 3 877 | 1.056 | 2.82× | 0.99× |
| | pre. $B$=4 $S$=4k | 50% | 45 | 7 026 | 3 914 | 3 895 | 4 395 | 3 918 | **3 886** | **3 886** | 3 874 | 1.002 | 1.81× | 1.00× |
| Llama-3-405B | dec. $B$=16,2k | 5% | 18 | 45 324 | 45 027 | 43 257 | 43 313 | 43 263 | 43 295 | **43 138** | 42 809 | 1.005 | 1.05× | 1.00× |
| | dec. $B$=16,2k | 5% | 45 | 18 294 | 18 000 | 17 288 | 17 476 | 17 459 | 17 306 | **17 272** | 17 124 | 1.010 | 1.06× | 1.00× |
| | pre. $B$=4 $S$=2k | 9% | 18 | 56 072 | 45 008 | 41 896 | 51 510 | 42 218 | 42 718 | **41 724** | 41 165 | 1.011 | 1.34× | 1.00× |
| | pre. $B$=4 $S$=2k | 9% | 45 | 28 966 | 18 019 | 16 797 | 27 065 | 16 919 | 22 387 | **16 744** | 16 466 | 1.014 | 1.73× | 1.00× |

Table 6: Constrained-cache configurations (a smaller model with 40% of its weights in HBM, as for a GPU shared with other services). Iteration latency in ms (mean of 10 seeds with common random numbers; best entry per row in bold). "Cache" is the share of weight bytes that fit in the HBM weight cache. FlexGen = per-layer fractional split with buffer count and lookahead tuned per row; Hot/Cold and Stride = equal-budget pinning with CAMP's or a tuned fixed lookahead (Section 5.3); LB = lower bound of Section 3.4; det./LB = noise-free CAMP-v2 latency divided by LB; "vs. best ext." = best of Demand/Static-$k$/FlexGen/Hot-Cold/CAMP-v1 divided by CAMP-v2.

| Model | Workload | Cache | GB/s | Demand | Static-$k$ | FlexGen | Hot/Cold | Stride | CAMP-v1 | CAMP-v2 | LB | det./LB | vs. Demand | vs. best ext. |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Llama-2-13B | dec. $B$=16,2k | 40% | 18 | 1 427 | 1 407 | **865** | 877 | 884 | 897 | 871 | 873 | 1.017 | 1.64× | 0.99× |
| | dec. $B$=16,2k | 40% | 45 | 585 | 565 | **349** | 360 | 356 | 361 | 352 | 349 | 1.024 | 1.66× | 0.99× |
| | pre. $B$=4 $S$=4k | 40% | 18 | 2 169 | 1 413 | **888** | 1 461 | 895 | 1 126 | 916 | 873 | 1.069 | 2.37× | 0.97× |
| | pre. $B$=4 $S$=4k | 40% | 45 | 1 314 | 758 | 754 | 976 | 753 | **751** | **751** | 746 | 1.005 | 1.75× | 1.00× |
| Gemma-2-9B | dec. $B$=32,2k | 40% | 18 | 1 026 | 1 010 | 674 | 706 | 713 | 810 | **631** | 633 | 1.013 | 1.63× | 1.07× |
| | dec. $B$=32,2k | 40% | 45 | 424 | 408 | 273 | 285 | 288 | 327 | **255** | 253 | 1.020 | 1.66× | 1.07× |

*Calibration is not decisive.* Replacing the calibrated cost model by the generic one changes CAMP-v2's latency by at most 4.7%, and by the exact device model by at most 0.7% (Section 6.6).

For Gemma-2-9B, CAMP-v2 is 1.07× faster than the tuned FlexGen-style split and 1.12× faster than frequency pinning at equal budget; we have not isolated the cause, but the likely reason is that the 1.8 GB tied embedding/head table dominates the unit sizes, so a uniform fractional split must reserve streaming buffers sized by it, whereas SCP can pin the table itself.

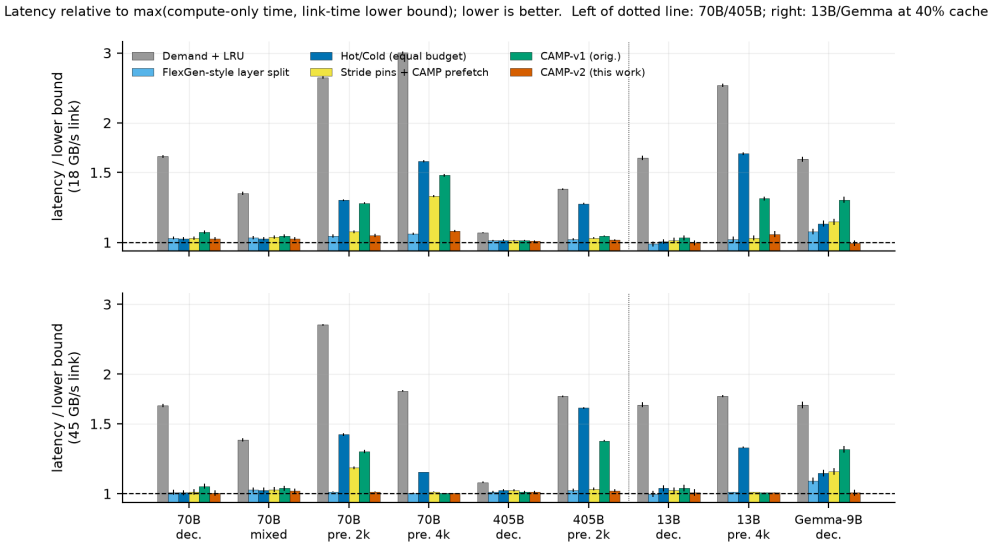



Figure 4: Iteration latency relative to max(compute-only time, link lower bound) at 18 and 45 GB/s (mean of 10 seeds, 95% confidence bars). Dashed line: lower bound. The FlexGen-style bar has its buffer count and lookahead tuned per configuration.

### *6.3. Which ingredients matter? Baseline attribution*

Figure 5 starts from a conventional hot/cold baseline (frequency pins with a two-slot reserve, position tie-break, lookahead 2, LRU) and adds one ingredient at a time. For 70B prefill with four 4k-token prompts at 18 GB/s the latencies relative to the starting point are 1.23 (equal-budget pinning, i.e. pinning more bytes), 1.03 (random tie-break), 1.03 (FIFO eviction and cross-iteration lookahead), 0.89 (interleaved stride pins), 0.84 (CAMP's prefetcher) and 0.69 (SCP pins); the tuned FlexGen-style split reaches 0.68. Three things follow. Pinning more bytes *without* interleaving makes the baseline slower (up to 1.38× across the nine rows), because a larger contiguous block lets the link run ahead of even fewer units. Interleaving is the largest single step for 13B prefill and 70B prefill with 2k-token prompts; for 70B prefill with 4k-token prompts the planner step is as large. CAMP's prefetcher is worth up to 1.06× over a fixed lookahead with stride pins, and SCP up to 1.22× over stride pins when both use CAMP's prefetcher. Over all nine rows the full policy is 1.00–1.45× faster than the conventional baseline; in link-bound decode every step is within a few per cent, as expected.

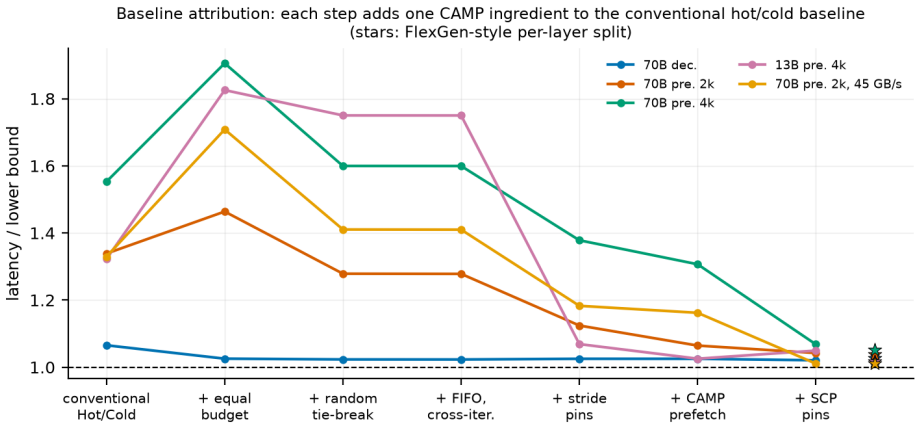



Figure 5: Baseline attribution: latency relative to the lower bound as each ingredient is added to a conventional hot/cold baseline. Stars: the tuned FlexGen-style split.

### *6.4. Pinning: what does the selection rule buy?*

Figure 6 compares pin-selection rules with the prefetcher fixed (CAMP's controller, next-use eviction): no pinning, prefix, frequency-ranked (conventional two-slot reserve, and equal budget with random tie-break), interleaved stride, the exact knapsack of Eq. (7), the marginal-gain greedy set, and SCP. Averaged over 14 configurations (models, workloads, two bandwidths) the latency relative to the lower bound is 1.431 without pinning, 1.122 for prefix, 1.119 for conventional frequency ranking, 1.188 for equal-budget frequency ranking, 1.187 for the knapsack, 1.065 for greedy marginal gain, 1.050 for stride and 1.018 for SCP (worst case 1.07). A resident set drawn at random with the same budget averages 1.17× SCP's latency (worst case 1.54×).

*Link-bound iterations are insensitive to the rule.* In decode every rule that pins about the same number of bytes lands within a few per cent of the others; what matters is the number of resident bytes.

*Balanced iterations are very sensitive to placement.* For 13B prefill with four 4k-token prompts at 18 GB/s the link utilisation is 80% for prefix pins and 63% for equal-budget frequency pins, which pin contiguous runs, 84% for the knapsack, and 100–98% for stride and SCP sets; the latencies are 1149, 1385, 1049, 895 and 916 ms. This is the mechanism of Eq. (10). The exact knapsack packs more bytes than frequency ranking but remains slower than interleaved sets because its benefit ignores overlap.

*SCP is robust rather than uniformly best.* Against the best of the six

Table 7: Optimality gap of the pin-set selection on an 18-unit model (8 layers, $d$=2048), where all $2^{18}$ pin sets can be enumerated. Planner gap: predicted iteration time of the selected set relative to the exhaustive optimum of the planner objective; the last column compares the SCP set with the planner-optimal set when both are replayed in the event engine (negative: SCP is faster there).

| **Workload** | **Cache ratio** | **SCP (%)** | **Stride (%)** | **Frequency (%)** | **SCP, simulated (%)** |
|---|---|---|---|---|---|
| decode $B$=16 | 0.3 | 0.0 | 14.1 | 17.7 | 0.0 |
| decode $B$=16 | 0.5 | 0.0 | 20.3 | 25.5 | -0.9 |
| decode $B$=16 | 0.7 | 0.0 | 0.4 | 32.4 | 0.0 |
| prefill $B$=1 $S$=2k | 0.3 | 0.0 | 12.5 | 16.1 | 0.0 |
| prefill $B$=1 $S$=2k | 0.5 | 0.0 | 18.8 | 24.0 | 0.0 |
| prefill $B$=1 $S$=2k | 0.7 | 0.5 | 0.8 | 31.0 | 1.2 |
| prefill $B$=4 $S$=4k | 0.3 | 0.0 | 9.7 | 13.0 | 1.0 |
| prefill $B$=4 $S$=4k | 0.5 | 0.0 | 13.2 | 17.8 | 2.2 |
| prefill $B$=4 $S$=4k | 0.7 | 3.8 | 5.9 | 22.4 | 12.6 |
| prefill $B$=8 $S$=2k | 0.3 | 0.0 | 9.7 | 13.0 | 0.3 |
| prefill $B$=8 $S$=2k | 0.5 | 0.0 | 13.2 | 17.8 | 0.9 |
| prefill $B$=8 $S$=2k | 0.7 | 4.2 | 5.2 | 22.8 | 12.0 |

simpler rules in each configuration, SCP lost at most 2.4% and gained more than 1% in 3 of 14 configurations, by up to 22.3%. No single simple rule is best across the configurations (stride in balanced prefill, frequency in encoder–decoder inference, none in some compute-bound cases); SCP selects among them, and finds sets outside them, without being told which applies.

*Optimality.* On an 18-unit model, where all $2^{18}$ pin sets can be enumerated, SCP is on average 0.71% (maximum 4.2%) above the exhaustive optimum of the planner objective over 12 cases, versus up to 20.3% for stride and 32.4% for frequency ranking (Table 7); replayed in the event engine the maximum gap of SCP is 12.6%, which marks the planner's approximation error that the dry-run verification reduces but does not remove. SCP is a heuristic with no optimality guarantee.

### *6.5. Non-uniform reuse and eviction*

*Eviction on a uniform loop.* Table 8 shows eviction policies on 13B decode with no pinning. LRU, FIFO and LFU produce identical results and zero hits (1428 ms): on a cyclic scan with equal access counts, LFU has no frequency signal to exploit and degenerates to LRU. Next-use eviction retains 33% of accesses (872 ms), because it is the only policy that keeps some layers at all.

*Workloads with genuinely different reuse.* Table 9 and Figure 7 give three cases where the number of accesses or the size per unit differs strongly between

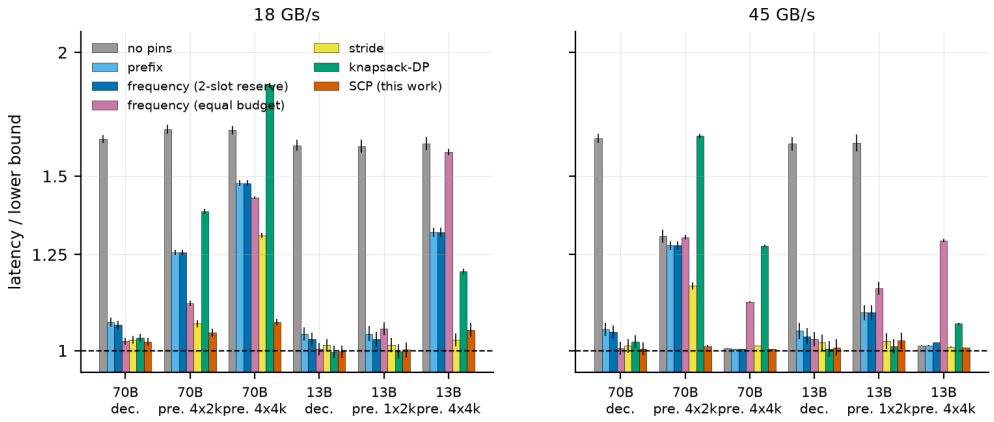



Figure 6: Pin-selection rules with the prefetcher and eviction fixed (latency relative to the lower bound; 10 seeds, 95% confidence bars). Placement matters when compute and transfer are comparable (prefill at 18 GB/s) and not in link-bound decode.

layers. For Gemma-2-9B, demand paging with LFU is faster than LRU (the tied table is accessed twice per step), and SCP is 1.62× faster than demand paging with LRU, 1.07× faster than the tuned FlexGen-style split and 1.12–1.28× faster than the pinning rules. In speculative decoding the draft model's layers are used four times per iteration; frequency pins already capture this (1.01× SCP's latency) while the FlexGen-style split and stride pins are 1.06× and 1.19×. For the encoder–decoder, encoder layers are used once per request and decoder layers 64 times: prefix pinning is useless (2.95×), LRU and LFU are identical (3.03×), the FlexGen-style split and stride pins, which are interleaved but blind to reuse, are 1.97× slower, while equal-budget frequency pins (1.00×) and the knapsack (1.09×) are close to SCP. Reuse awareness is therefore decisive when per-layer frequencies differ by large factors, and interleaving alone is not a substitute for it; but the simplest reuse-aware rule (frequency at equal budget) is as good as SCP there. The distinctive property of SCP is that one mechanism gets all three cases right.

### *6.6. How much does the compute estimate matter?*

Table 10 and Fig. 8 address how $\hat{t}_u$ is obtained and what happens when it is wrong. The generic roofline estimate has a mean absolute error of 23.2% (13B prefill), 23.5% (70B prefill) and 9.2% (70B decode) against the simulated device; one probe iteration reduces this to 1.6, 0.1 and 0.1%. Yet the latency of CAMP-v2 with SCP planned from the generic, the calibrated or the exact

Table 8: Eviction policies on a uniform dense decode loop (Llama-2-13B, $B$=16, 2k context, 40% cached, 18 GB/s, no pinning). On a cyclic access pattern with equal frequencies, LRU, FIFO and LFU are indistinguishable (zero hits); only next-use eviction with the known layer order retains useful layers.

| | **demand paging** | | **prefetch, depth 2** | |
|---|---|---|---|---|
| **Eviction** | latency (ms) | hit (%) | latency (ms) | hit (%) |
| LRU | 1 428 | 0.0 | 1 407 | 0.2 |
| FIFO | 1 428 | 0.0 | 1 407 | 0.2 |
| LFU | 1 428 | 0.0 | 1 407 | 0.2 |
| Random | 1 299 | 7.3 | 1 264 | 9.3 |
| Next-use (graph-aware) | 872 | 32.5 | 861 | 32.7 |

Table 9: Workloads with non-uniform reuse or layer sizes (latency in seconds; 18 GB/s; 35–40% of weights cached; best in bold, ties within 0.5%). The first three columns are not CAMP variants; the remaining columns use CAMP's prefetcher with the stated pinning rule. Equal-budget rules keep one streaming slot free; "prefix" uses the conventional two-slot reserve.

| **Workload** | **LRU** | **LFU** | **FlexGen** | **none** | **prefix** | **freq.** | **stride** | **DP** | **SCP** |
|---|---|---|---|---|---|---|---|---|---|
| Gemma-2-9B, tied embedding (per step) | 1.03 | 0.93 | 0.68 | 1.01 | 0.81 | 0.71 | 0.72 | 0.72 | **0.63** |
| speculative decoding, 13B+1B, $k$=4 (per step) | 1.53 | 1.43 | 1.07 | 1.51 | 1.03 | 1.02 | 1.20 | 1.03 | **1.01** |
| encoder–decoder, 64 tokens (per request) | 27.77 | 27.77 | 18.06 | 27.17 | 27.02 | **9.14** | 18.02 | 9.95 | **9.16** |

model differs by at most 0.4% in these workloads, whereas removing pinning altogether costs 1.54–1.61×. Prefetching is insensitive to the estimate (the work-conserving controller and the earlier compute-gated one give identical latencies). The estimate can mislead the pinning stage if it is *systematically biased*: with a static estimator whose constants are off by factors of 0.25 to 4, SCP's latency is up to 10.7% (13B prefill) and 22.6% (70B prefill) above the best; after online calibration the worst case over the bias range is 2.4% and 0.2%. Decode is insensitive throughout (at most 1.0%) because it is link-bound.

A tighter test is a ground truth that the estimator cannot represent. Table 11 makes the simulated GEMM efficiency ramp with arithmetic intensity, which the estimator ignores. The generic estimate then has a mean error of up to 24%; calibrating on the evaluated shape reduces it to 0.1%, and calibrating on a different shape (decode probe, prefill evaluation or vice versa) leaves up to 13%, yet the latency of the resulting plan differs from the oracle plan by at most 0.4%. In summary, CAMP needs the compute estimate only

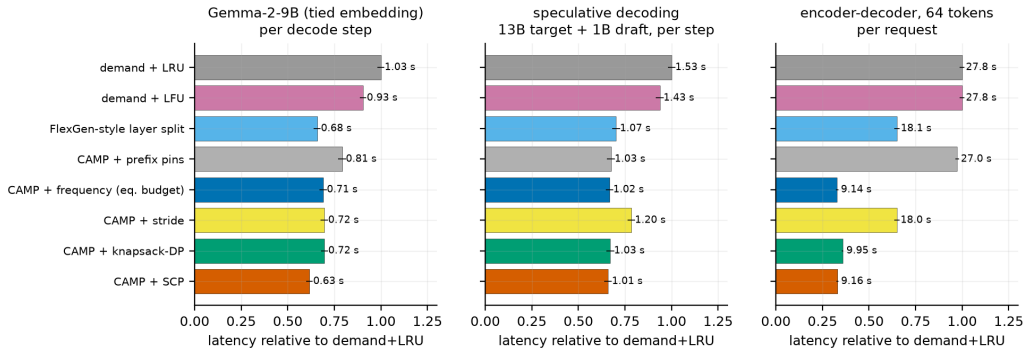



Figure 7: Workloads with non-uniform reuse (18 GB/s; 10 seeds). Bars: latency relative to demand+LRU; absolute value at the bar end.

Table 10: Runtime cost model. Mean absolute percentage error (MAPE) of the per-layer compute-time estimate before and after online calibration with one probe iteration, and the resulting iteration latency (ms) when SCP pins are planned without pinning, from the generic roofline estimate, from the calibrated estimate and from the exact device model.

| **Workload** | **MAPE generic (%)** | **MAPE calib. (%)** | **no pins** | **generic** | **calib.** | **exact** | **LB** |
|---|---|---|---|---|---|---|---|
| llama2-13b prefill B=4 S=4k | 23.2 | 1.6 | 1 410 | 914 | 915 | 916 | 873 |
| llama3-70b prefill B=4 S=2k | 23.5 | 0.1 | 6 214 | 3 864 | 3 873 | 3 857 | 3 728 |
| llama3-70b decode B=16 | 9.2 | 0.1 | 7 767 | 4 848 | 4 858 | 4 858 | 4 772 |

for pinning, only in the balanced regime, and an exponential moving average over one iteration of observed layer times is enough; we do not consider the cost model a central contribution.

### *6.7. Batching, model size and link sensitivity*

*Batching.* Figure 9 shows throughput for Llama-3-70B. Panel (a) varies the number of tokens per iteration at fixed cache: throughput grows linearly until about 16k tokens per iteration, where it saturates at the compute-bound limit. Panel (b) is the realistic decode case, in which a larger batch also needs a larger KV cache: the weight cache is the HBM left after $B$ sequences of 4k tokens. The cached share of the weights shrinks from 54% at $B = 1$ to 23% at $B = 256$, and $B = 512$ no longer fits at all. Throughput therefore grows more slowly than linearly: the slowdown relative to resident weights rises from 70× at $B = 1$ to a maximum of 81×, and at $B = 256$ the system serves 42 tokens/s against 3339 for hypothetical resident weights. Streaming amortises weight traffic over the batch, but the KV cache that a large batch needs competes with the weights for HBM.

Table 11: Cost-model error under a shape-dependent ground truth (GEMM efficiency ramps with arithmetic intensity, which the runtime estimator does not model), Llama-3-70B, 18 GB/s. "Probe" is the shape on which the online calibration is performed; a probe from a different shape leaves a residual error, and the plan is evaluated on the other shape.

| Evaluated on | Probe | Estimator | MAPE (%) | Latency (ms) | LB (ms) |
|---|---|---|---|---|---|
| decode $B$=16 | decode $B$=16 | generic (no calibration) | 9.2 | 4 848 | 4 772 |
| decode $B$=16 | decode $B$=16 | calibrated on probe shape | 0.1 | 4 858 | 4 772 |
| decode $B$=16 | decode $B$=16 | constants-exact (no ramp knowledge) | 0.0 | 4 858 | 4 772 |
| decode $B$=16 | prefill $B$=4 $S$=2k | calibrated on probe shape | 11.8 | 4 858 | 4 772 |
| prefill $B$=4 $S$=2k | decode $B$=16 | calibrated on probe shape | 13.2 | 3 855 | 3 728 |
| prefill $B$=4 $S$=2k | prefill $B$=4 $S$=2k | generic (no calibration) | 23.5 | 3 864 | 3 728 |
| prefill $B$=4 $S$=2k | prefill $B$=4 $S$=2k | calibrated on probe shape | 0.1 | 3 873 | 3 728 |
| prefill $B$=4 $S$=2k | prefill $B$=4 $S$=2k | constants-exact (no ramp knowledge) | 0.0 | 3 857 | 3 728 |

*Model size.* Figure 10 moves from 7B to 405B parameters with a fixed 80 GB HBM. The 7B model fits and every policy is equal. For 13B decode with 16 sequences, 86% of the weights stay resident and CAMP (211 ms at 18 GB/s) is 1.09× *slower* than the tuned FlexGen-style split (194 ms) and above the whole-unit lower bound (200 ms), which the fractional split can undercut by up to 3.1%; demand paging takes 1429 ms. The likely cause, which we did not isolate, is the granularity of whole units: with 86% resident, the streaming slot must hold the largest unit. At 70B (39% cached) decode takes 4870 ms at 18 GB/s; at 405B (5% cached) it takes 43.2 s, 148× slower than resident weights at 18 GB/s and 59× at 45 GB/s, with prefill at 3.83× and 1.53×. At 405B almost nothing can stay resident, so demand paging is only 1.05× slower than CAMP in the least favourable decode row.

*Link, cache and path.* Figure 11 sweeps $BW_{\text{eff}}$ from 8 to 64 GB/s for 70B. In decode, CAMP follows the lower bound within 1.018× over the whole range and the Hot/Cold and FlexGen-style baselines are within 1.006 and 1.010× of CAMP. In balanced prefill (4k-token prompts) CAMP is at most 1.07× the bound (at 18 GB/s) and 1.003× at 64 GB/s; Hot/Cold is 1.08–1.50× slower than CAMP across the range, and the FlexGen-style split is 0.986–1.005×. Sweeping the cache share from 10% to 100% at 18 GB/s keeps CAMP within 1.07× of the bound (the FlexGen-style split within 1.051, Hot/Cold up to 1.60). The staged path is a large penalty: with bounce-buffer copies the effective bandwidth drops to 5.5 GB/s (from 18) and 6.8 GB/s (from 45), and CAMP's latency rises 2.7–6.7×, so page-locking the CXL region is essential.

### *6.8. Several GPUs behind one expander*

A pooled expander serves several GPUs through one root complex, so its bandwidth is divided among the GPUs that stream at the same time.

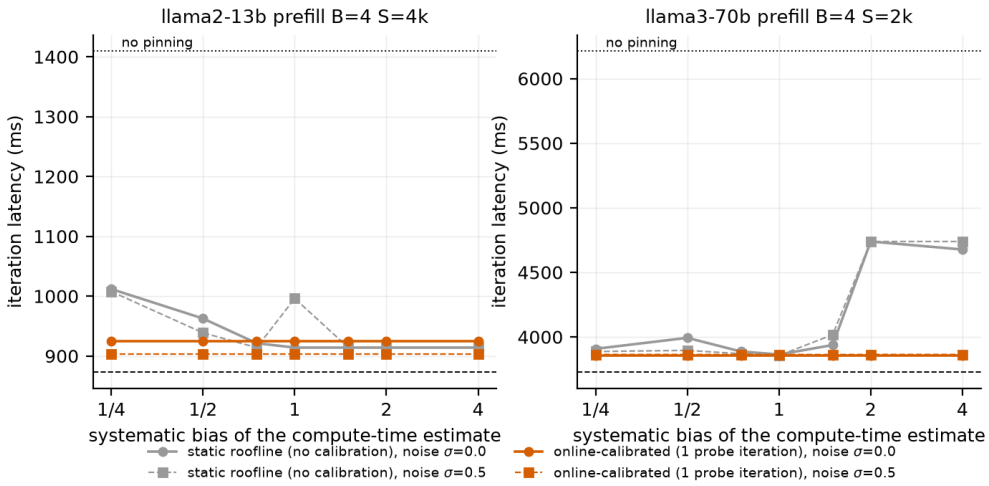



Figure 8: Iteration latency when SCP is planned from a compute estimate with systematic bias (x-axis) and per-layer noise, with and without online calibration. Dotted: no pinning; dashed: lower bound.

We model the worst case, in which $G$ identical GPUs stream simultaneously and each receives $BW_{\text{eff}}/G$, and run CAMP-v2 per GPU (70B, 40% of the weights resident on each GPU). Because $T^\star$ in Eq. (3) is inversely proportional to bandwidth, sharing multiplies it by $G$, and streaming remains nearly free while $G \le T/T^\star$ for the pool bandwidth. Table 12 confirms this. With an 18 GB/s pool and 64k tokens per iteration ($T^\star = 20.4$k) two GPUs run at 1.01× of resident speed and four at 1.10×; with a 45 GB/s pool and 16k tokens ($T^\star = 8.2$k) two GPUs run at 1.01× and four at 1.73×. At 64k tokens, 45 GB/s feeds eight GPUs at 1.01× and sixteen at 1.72×. Where streaming is transfer-bound the measured slowdown is 0.85–1.00× of $GT^\star/T$ (median 0.88), so the rule is slightly conservative because $T^\star$ counts only projection FLOPs at peak efficiency, and CAMP stays within 1.067× of the lower bound at every sharing level. For decode ($T \le 512$) no feasible $G$ exists, which restates the main finding: a shared expander helps only compute-dense iterations. Equal sharing and simultaneous streaming are the worst case; staggering the GPUs' iterations or skewing the shares would relax it, and we did not model arbitration in the root complex.

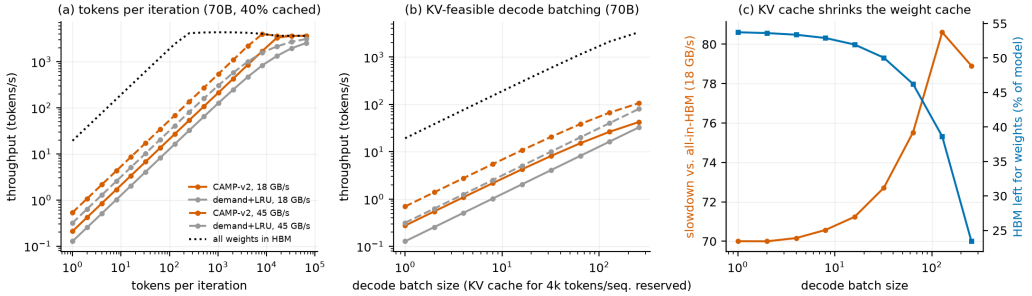



Figure 9: Throughput versus batch size, Llama-3-70B. (a) Tokens per iteration (GEMM batch, 40% of weights cached). (b) KV-feasible decode batch size (KV cache for 4k tokens per sequence reserved; infeasible batches omitted). (c) Slowdown versus resident weights and the share of weights that fit in HBM.

Table 12: Several GPUs behind one expander (70B, 40% of the weights resident per GPU, all GPUs streaming at once, CAMP-v2). Slowdown relative to resident weights when the pool bandwidth is shared equally among $G$ GPUs. $T^\star$ is the threshold of Eq. (3) for the whole pool; $T/T^\star$ is the predicted largest $G$ with near-free streaming.

| **Pool GB/s** | $T$ | $T^\star$ | $T/T^\star$ | $G$=1 | $G$=2 | $G$=4 | $G$=8 | $G$=16 |
|---|---|---|---|---|---|---|---|---|
| 18 | 16 384 | 20.4k | 0.8 | 1.10 | 2.16 | 4.24 | 8.50 | 16.9 |
| 18 | 65 536 | 20.4k | 3.2 | 1.01 | 1.01 | 1.10 | 2.15 | 4.27 |
| 45 | 16 384 | 8.2k | 2.0 | 1.00 | 1.01 | 1.73 | 3.40 | 6.80 |
| 45 | 65 536 | 8.2k | 8.0 | 1.00 | 1.00 | 1.01 | 1.01 | 1.72 |

### *6.9. Alternatives to streaming*

Streaming from CXL competes with other ways to run a model that does not fit. Table 13 compares it with weight quantisation, pinned host DRAM and two GPUs (all other columns use CAMP-v2). Quantisation is the strongest alternative where its accuracy cost is acceptable: FP8/INT8 weights make 70B decode 5.7× faster over the same 18 GB/s link (854 versus 4875 ms) and 70B prefill fits one GPU, and INT4 weights fit entirely; for 405B even INT4 weights (203 GB) need streaming and decode takes 9.1 s. Pinned host DRAM at 52 GB/s is the same mechanism over a faster link (70B decode 1656 ms versus 4875 ms at 18 GB/s and 1912 ms at 45 GB/s). For FP16 70B prefill, streaming takes 3795 ms at 18 GB/s against an optimistic 943 ms on two GPUs (perfect scaling, no communication). Streaming from CXL is therefore attractive when quantisation is not acceptable, when the model exceeds the GPU count available, or when the iteration is compute-dense; it

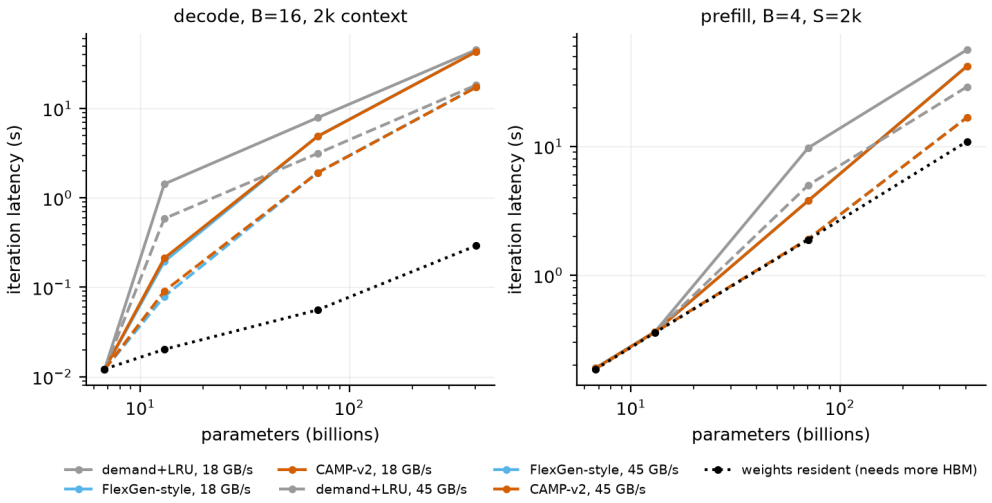



Figure 10: Iteration latency versus model size at fixed 80 GB HBM (B=16 decode with 2k context; B=4, S=2k prefill). The dotted line is the latency if all weights were resident.

does not compete with a second GPU on latency.

### *6.10. Robustness of the conclusions*

Figure 12a scales jitter, drift and compute noise together from zero to eight times the default (log-normal copy jitter up to $\sigma = 0.5$, drift up to 0.4, compute noise up to 0.25). The ratio between CAMP-v2 and the tuned FlexGen-style split moves from 0.995 (decode) and 1.002 (prefill) without noise to 0.993 and 1.026 at the highest level; against demand paging it ends at 0.64 and 0.42. The ordering and the size of the gains are therefore not artefacts of a noise level. Figure 12b shows the planner against the deterministic engine for 80 random configurations (five models, decode and prefill, cache shares of 15–70%, six link rates): mean absolute error 0.18%, median 0.01%, maximum 3.3%. Figure 12c plans SCP with a mis-specified link model (assumed bandwidth 0.7–1.3 times the true one, per-copy latency 1 or 4 times): latency changes by at most 1.3%, so the plan does not require an accurate link calibration.

## 7. Discussion and limitations

*What the results mean for CXL-backed inference.* With an effective bandwidth of 10–20 GB/s per expander, streaming weights from CXL is practical for

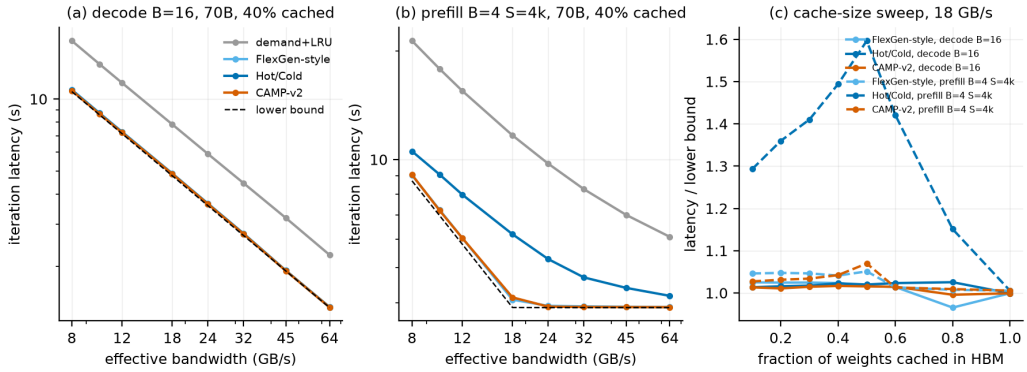



Figure 11: (a, b) Iteration latency versus effective bandwidth for decode and balanced prefill (70B). (c) Latency relative to the lower bound versus the cached fraction of the weights at 18 GB/s.

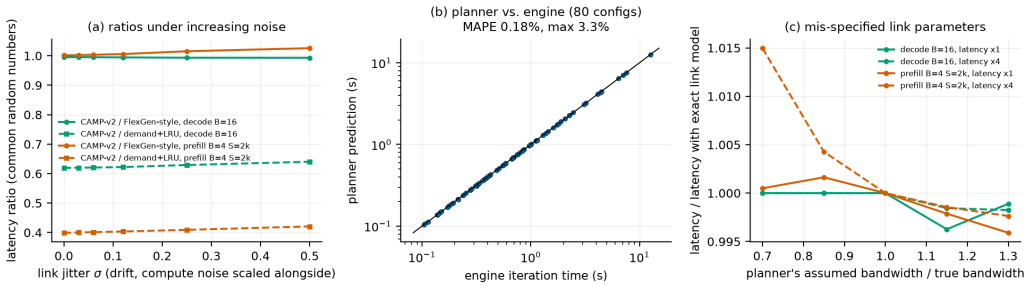



Figure 12: (a) Latency ratios of CAMP-v2 to baselines under increasing noise (paired seeds). (b) Pipeline planner versus deterministic event engine. (c) SCP planned with a mis-specified link model.

compute-dense iterations (long-prompt prefill, very large batches) and impractical for interactive decode, where the slowdown against resident weights is tens of times and prefetching cannot help because the iteration is link-bound. For decode the achievable gains come from the number of resident bytes and their placement, and from raising $BW_{\text{eff}}$ (more expanders, page-locked DMA, compression, quantisation). Once placement is interleaved, policy sophistication buys little on dense models with uniform layers: a tuned FlexGen-style split is within a few per cent of CAMP. What CAMP adds is (i) not needing that tuning, (ii) a verified plan whose quality is known against a lower bound, and (iii) correct handling of non-uniform unit sizes and reuse, where uniform splits and stride rules lose up to a factor of two. Whether (i)–(iii) justify a

Table 13: Alternatives to streaming from a CXL expander: weight quantisation, pinned host DRAM, and two GPUs with tensor parallelism (latency in ms; CAMP-v2 where weights are streamed; "resident" = all weights fit one 80 GB GPU together with the KV cache). The two-GPU column is an optimistic estimate (perfect scaling, no communication) and is shown only to indicate when a second GPU makes streaming unnecessary.

| Model | Weights | Phase | GB | 1 GPU fits | CXL 18 | CXL 45 | host DRAM 52 | 2 GPUs (opt.) |
|---|---|---|---|---|---|---|---|---|
| Llama-3-70B | FP16 | dec. | 141 | no | 4 875 | 1 912 | 1 656 | does not fit |
| Llama-3-70B | FP16 | pre. | 141 | no | 3 795 | 1 906 | 1 906 | 943 |
| Llama-3-70B | FP8/INT8 | dec. | 71 | no | 854 | 346 | 302 | 16 |
| Llama-3-70B | FP8/INT8 | pre. | 71 | yes | 1 886 | 1 886 | 1 886 | 943 |
| Llama-3-70B | INT4 | dec. | 35 | yes | 20 | 20 | 20 | 10 |
| Llama-3-70B | INT4 | pre. | 35 | yes | 1 886 | 1 886 | 1 886 | 943 |
| Llama-3-405B | FP16 | dec. | 812 | no | 43 206 | 17 319 | 15 087 | does not fit |
| Llama-3-405B | FP16 | pre. | 812 | no | 41 788 | 16 820 | 14 641 | does not fit |
| Llama-3-405B | FP8/INT8 | dec. | 406 | no | 20 431 | 8 307 | 7 193 | does not fit |
| Llama-3-405B | FP8/INT8 | pre. | 406 | no | 18 948 | 10 929 | 10 929 | does not fit |
| Llama-3-405B | INT4 | dec. | 203 | no | 9 149 | 3 624 | 3 119 | does not fit |
| Llama-3-405B | INT4 | pre. | 203 | no | 10 996 | 10 921 | 10 921 | does not fit |

more complex runtime is a deployment decision; our results say the gain is modest for dense models and large for non-uniform ones.

*Cost of re-planning.* Changing the resident set requires moving up to the pinned bytes once. At 18 GB/s that is 2.9 s for the 53 GB pinned for 70B decode, 3.9 s for the 70 GB pinned for 70B prefill, and 0.6 s for 10 GB for 13B decode. Relative to running the same workload without pinning, the move pays for itself after about 1.0, 1.7 and 1.0 iterations respectively, so re-planning is worthwhile only when the operating point persists for more than a few iterations. The move is not charged in the experiments.

*Limitations.*

1. *No CXL hardware.* The CXL link is simulated; its bandwidth and fixed costs are taken from published measurements [6, 7] and swept over a wide range, and the link and overlap logic is checked on a PCIe-attached T4 with pinned host DRAM in place of a CXL pool. We did not measure a real expander, a PCIe Gen5 GPU or an H100; the compute times of the H100-class device come from a roofline model with efficiencies that we did not measure.
2. *Partly unexplained hardware errors.* The check on the T4 shows errors of up to 25% where compute and copy are comparable, with a sign that differs between regimes, and a closed-form expression is as accurate as the simulator there. We have not isolated the cause. These are exactly the regimes in which placement matters, so the quantitative gains in the balanced regime carry this uncertainty.

3. *Baselines are reimplementations* of mechanisms, not ports of FlexGen, PowerInfer, TMO or Limoncello. The FlexGen-style baseline is tuned per configuration by simulation and modelled at fractional layer granularity, which is more favourable to it than whole-unit placement; conversely we did not implement FlexGen's full LP, which also places KV cache and activations. Reusing the same simulator for all policies removes implementation noise but also shares any modelling error among them.
4. *Scope of the models.* We evaluate dense transformers with a static order. Mixture-of-Experts routing makes the access order data-dependent; hot/cold partitioning by observed frequency [23, 25] is the natural baseline and an online variant of SCP is future work.
5. *Pool sharing is idealised.* Section 6.8 divides the pool bandwidth equally among simultaneously streaming GPUs; root-complex arbitration, unequal tenants, CXL switches and contention with host memory traffic are not modelled.
6. *The KV cache stays in HBM.* Offloading it to CXL [6, 28] competes for the same link, and Section 6.7 shows how strongly the KV cache already competes with the weights for HBM.
7. *SCP is a heuristic.* It is within 4.2% of the exhaustive optimum of its own objective on an 18-unit model, but there is no guarantee for larger models, and in the event engine the set it selects is up to 12.6% slower than the planner-optimal set; the dry-run replay mitigates but does not remove this.
8. *Whole-unit granularity.* CAMP places whole attention or feed-forward blocks. With a high resident share (13B decode, 86% resident) a sub-layer split is better by 1.09× (Section 6.7); applying SCP to tensor-level units is a straightforward extension that we did not evaluate.
9. *Heterogeneous workloads are synthetic where noted.* The encoder–decoder and speculative-decoding workloads are constructed from model-card configurations, not traces of a deployed system.

## 8. Conclusion

We studied LLM inference with weights streamed from CXL expanders through a host-mediated DMA path. A closed-form threshold and a regime map show when streaming is nearly free (above about 16k tokens per iteration) and when it is tens of times slower (decode). Which layers are resident matters

less than where they sit: contiguous resident sets idle the link when compute and transfer are comparable, interleaved ones do not, and a per-layer fractional split in the style of FlexGen is already interleaved. CAMP, which combines a work-conserving prefetcher with stall-cost-aware pinning, stays within 1.014–1.069× of an implementation-independent lower bound in simulation, is up to 2.82× faster than demand paging, ties the best baseline on dense uniform models and is ahead where unit sizes or reuse are non-uniform; the compute estimate and the LFU/LRU distinction, often presented as central, matter little. The link and overlap logic of the simulator was checked on a real GPU with host memory standing in for the pool, with mixed accuracy that we report as is. Sharing one expander among $G$ GPUs raises the threshold $G$-fold, so one expander feeds $G \leq T/T^{\star}$ GPUs at near-resident speed. The main open items are measurement on real CXL hardware and PCIe Gen5 GPUs, tensor-level pinning, online variants for Mixture-of-Experts, and multi-tenant sharing of the pool.

## CRediT authorship contribution statement

**Quang-Vinh Dang**: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization, Supervision. **Dat Le**: Validation, Writing – review & editing. **Minh Ngoc Dinh**: Validation, Writing – review & editing.

## Data availability

The simulator, experiment scripts, raw result files, the hardware microbenchmark and its measured output are available at `https://github.com/vinhqdang/Enabling-CXL-Based-Memory-Disaggregation-for-Large-Scale-AI-Systems`. Every figure, table and number in the text is generated from the files in `results/` by `experiments/plots.py`, `experiments/make_tables.py` and `experiments/make_numbers.py`.

## Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used generative AI-assisted tools for software implementation, experiment scripting, analysis scripts and drafting and editing of text. After using these tools, the authors reviewed, verified and edited all code, results and content as needed and take full responsibility for the content of the publication.

## References


[1] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, C. Zhang, FlexGen: High-throughput generative inference of large language models with a single GPU, in: Proceedings of the 40th International Conference on Machine Learning (ICML '23), 2023. URL `https://proceedings.mlr.press/v202/sheng23a.html`

[2] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, Y. He, ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC), 2021.

[3] K. Alizadeh, S. I. Mirzadeh, D. Belenko, S. K. Khatamifard, M. Cho, C. C. Del Mundo, M. Rastegari, M. Farajtabar, LLM in a flash: Efficient large language model inference with limited memory, in: Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL), 2024, pp. 12562–12584.

[4] H. Li, D. S. Berger, L. Hsu, D. Ernst, P. Zardoshti, S. Novakovic, M. Shah, S. Rajadnya, S. Lee, I. Agarwal, M. D. Hill, M. Fontoura, R. Bianchini, Pond: Cxl-based memory pooling systems for cloud platforms, in: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS '23, ACM, New York, NY, USA, 2023, pp.

574–587. doi:10.1145/3575693.3578835.
URL https://doi.org/10.1145/3575693.3578835

[5] H. A. Maruf, H. Wang, A. Dhanotia, J. Weiner, N. Agarwal, P. Bhattacharya, C. Petersen, M. Chowdhury, S. Kanaujia, P. Chauhan, Tpp: Transparent page placement for cxl-enabled tiered-memory, in: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS '23, ACM, New York, NY, USA, 2023, pp. 742–755. doi:10.1145/3582016.3582063.
URL https://doi.org/10.1145/3582016.3582063

[6] D. Yoon, Y. Min, H. Kim, S. H. Noh, J. Kim, TraCT: Disaggregated LLM serving with CXL shared memory KV cache at rack-scale (2025). arXiv:2512.18194.

[7] H. Jang, Y. Min, S. Kim, T. Ahn, H. Kim, Y. Joo, H. Kim, J. Kim, ITME: Inference tiered memory expansion with disaggregated CXL-hybrid memories (2026). arXiv:2606.12556.

[8] M. Rhu, N. Gimelshein, J. Clemons, A. Zulfiqar, S. W. Keckler, vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design, in: Proc. 49th IEEE/ACM Int. Symp. on Microarchitecture (MICRO), 2016, pp. 1–13.

[9] C.-C. Huang, G. Jin, J. Li, SwapAdvisor: Pushing deep learning beyond the GPU memory limit via smart swapping, in: Proc. 25th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2020, pp. 1341–1355.

[10] M. Hildebrand, J. Khan, S. Trika, J. Lowe-Power, V. Akella, AutoTM: Automatic tensor movement in heterogeneous memory systems using integer linear programming, in: Proc. 25th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2020, pp. 875–890.

[11] X. Peng, X. Shi, H. Dai, H. Jin, W. Ma, Q. Xiong, F. Yang, X. Qian, Capuchin: Tensor-based GPU memory management for deep learning, in: Proc. 25th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2020, pp. 891–905.

[12] J. Ren, J. Luo, K. Wu, M. Zhang, H. Jeon, D. Li, Sentinel: Efficient tensor migration and allocation on heterogeneous memory systems for deep learning, in: Proc. IEEE Int. Symp. on High-Performance Computer Architecture (HPCA), 2021, pp. 598–611.

[13] D. Gouk, S. Lee, M. Kwon, M. Jung, Direct access, high-performance memory disaggregation with directcxl, in: 2022 USENIX Annual Technical Conference, USENIX ATC '22, USENIX Association, Berkeley, CA, USA, 2022, pp. 287–294.
URL https://www.usenix.org/conference/atc22/presentation/gouk

[14] J. Liu, H. Hadian, Y. Wang, D. S. Berger, M. Nguyen, X. Jian, S. H. Noh, H. Li, Systematic cxl memory characterization and performance analysis at scale, in: Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS '25, ACM, New York, NY, USA, 2025, pp. 404–418. doi:10.1145/3676641.3715987.
URL https://doi.org/10.1145/3676641.3715987

[15] Y. Sun, Y. Yuan, Z. Yu, R. Kuper, C. Song, J. Huang, H. Ji, S. Agarwal, J. Lou, I. Jeong, R. Wang, J. H. Ahn, T. Xu, N. S. Kim, Demystifying cxl memory with genuine cxl-ready systems and devices, in: Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO '23, ACM, New York, NY, USA, 2023, pp. 105–121. doi:10.1145/3613424.3614256.
URL https://doi.org/10.1145/3613424.3614256

[16] H. Kim, N. Wang, Q. Xia, J. Huang, A. Yazdanbakhsh, N. S. Kim, LIA: A single-GPU LLM inference acceleration with cooperative AMX-enabled CPU–GPU computation and CXL offloading, in: Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA), 2025, pp. 544–558. doi:10.1145/3695053.3731092.

[17] S. Lin, Z. Guo, J. Lin, DAK: Direct-access-enabled GPU memory offloading with optimal efficiency for LLM inference (2026). arXiv:2604.26074.

[18] Z. Wang, S. Mahar, L. Li, J. Park, J. Kim, T. Michailidis, Y. Pan, M. Shen, T. Rosing, D. Tullsen, S. Swanson, J. Zhao, The hitchhiker's guide to programming and optimizing cache coherent heterogeneous

systems: CXL, NVLink-C2C, and AMD infinity fabric (2024). arXiv:2411.02814.

[19] R. Xie, A. U. Haq, Y. Fang, L. Ma, Z. B. Engineer, L. Liu, T. Zhang, TRACE: Unlocking effective CXL bandwidth via lossless compression and precision scaling (2025). arXiv:2509.03377.

[20] Y. Wang, L. Wu, W. Hong, Y. Ou, Z. Wang, S. Gao, J. Zhang, S. Ma, D. Dong, X. Qi, M. Lai, N. Xiao, CXL-DMSim: A full-system CXL disaggregated memory simulator with comprehensive silicon validation, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (2025).

[21] G. Gerganov, contributors, llama.cpp: LLM inference in C/C++, https://github.com/ggml-org/llama.cpp (2023).

[22] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, I. Stoica, Efficient memory management for large language model serving with PagedAttention, in: Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23), ACM, New York, NY, USA, 2023.
URL https://arxiv.org/abs/2309.06180

[23] Y. Song, Z. Mi, H. Xie, H. Chen, PowerInfer: Fast large language model serving with a consumer-grade GPU, in: Proceedings of the 30th Symposium on Operating Systems Principles (SOSP '24), ACM, New York, NY, USA, 2024, pp. 590–606. doi:10.1145/3694715.3695964.
URL https://arxiv.org/abs/2312.12456

[24] K. Kamahori, T. Tang, Y. Gu, K. Zhu, B. Kasikci, Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models, in: Proc. Int. Conf. on Learning Representations (ICLR), 2025.

[25] L. Xue, Y. Fu, Z. Lu, L. Mai, M. Marina, MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache, arXiv:2401.14361 (2024).

[26] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, B.-G. Chun, Orca: A distributed serving system for transformer-based generative models, in: 16th USENIX Symposium on Operating Systems Design and Implementation

(OSDI '22), USENIX Association, 2022.
URL https://www.usenix.org/conference/osdi22/presentation/yu

[27] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, R. Ramjee, Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve, in: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2024.

[28] W. Lee, J. Lee, J. Seo, J. Sim, InfiniGen: Efficient generative inference of large language models with dynamic KV cache management, in: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24), USENIX Association, Santa Clara, CA, USA, 2024.
URL https://www.usenix.org/conference/osdi24/presentation/lee-wonbeom

[29] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, X. Xu, Mooncake: Trading more storage for less computation—A KVCache-centric architecture for serving LLM chatbot, in: 23rd USENIX Conference on File and Storage Technologies (FAST '25), USENIX Association, Santa Clara, CA, USA, 2025, pp. 155–170, best Paper Award.
URL https://arxiv.org/abs/2407.00079

[30] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, H. Zhang, DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving, in: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24), USENIX Association, Santa Clara, CA, USA, 2024.
URL https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

[31] L. A. Belady, A study of replacement algorithms for a virtual-storage computer, IBM Systems Journal 5 (2) (1966) 78–101.

[32] R. Chen, Z. Ding, S. Zheng, C. Zhang, J. Leng, X. Liu, Y. Liang, MAGIS: Memory optimization via coordinated graph transformation and scheduling for DNN, in: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS '24, ACM, La Jolla, CA, USA,

2024, pp. 607–621. doi:10.1145/3620666.3651330.
URL https://doi.org/10.1145/3620666.3651330

[33] P. Zhao, H. Zhang, F. Fu, X. Nie, Q. Liu, F. Yang, Y. Peng, D. Jiao, S. Li, J. Xue, Y. Tao, B. Cui, MEMO: Fine-grained tensor management for ultra-long context LLM training, Proceedings of the ACM on Management of Data 3 (1) (2025). doi:10.1145/3709703.
URL https://arxiv.org/abs/2407.12117

[34] J. Weiner, N. Agarwal, D. Schatzberg, L. Yang, H. Wang, B. Sanouillet, B. Sharma, T. Heo, M. Jain, C. Tang, D. Skarlatos, TMO: Transparent memory offloading in datacenters, in: Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2022, pp. 609–621. doi:10.1145/3503222.3507731.

[35] A. Jain, H. Lin, C. Villavieja, B. Kasikci, C. Kennelly, M. Hashemi, P. Ranganathan, Limoncello: Prefetchers for scale, in: R. Gupta, N. B. Abu-Ghazaleh, M. Musuvathi, D. Tsafrir (Eds.), Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024, ACM, 2024, pp. 577–590. doi:10.1145/3620666.3651373.
URL https://doi.org/10.1145/3620666.3651373

[36] A. Agrawal, N. Kedia, J. Mohan, A. Panwar, N. Kwatra, B. Gulavani, R. Ramjee, A. Tumanov, VIDUR: A large-scale simulation framework for LLM inference, in: Proceedings of Machine Learning and Systems (MLSys), 2024.

[37] T. Sun, T. He, B. Ji, P. Parag, Optimizing resource allocation for geographically-distributed inference by large language models, Performance Evaluation 170 (2025) 102527.

[38] S. Williams, A. Waterman, D. Patterson, Roofline: An insightful visual performance model for multicore architectures, Communications of the ACM 52 (4) (2009) 65–76.